

CONST ROLE = SENIOR BACKEND ENGINEER

JavaScript & Node.js, Deeply Understood.

The caveats, internals, and production judgment that separate a senior engineer from someone who just knows the syntax — plus 90+ real interview questions with full explanations.

EVENT LOOP & ASYNC

MEMORY & GC

STREAMS & BACKPRESSURE

SCALING & RESILIENCE

INTERVIEW-READY

For engineers who ship to
production, not just to `localhost`

</> BACKEND MASTERY SERIES

SENIOR BACKEND ENGINEER SERIES

JavaScript & Node.js, Deeply Understood

Production-grade internals, caveats, and interview-tested judgment for engineers who own backend systems in the MERN and Node ecosystem.

Table of Contents

Three parts: JavaScript internals, Node.js production engineering, and a 90+ question interview bank.

PART I — JAVASCRIPT FOR THE BACKEND

01	The Event Loop, Microtasks & Macrotasks	08
02	Closures, Scope & Memory Leaks	10
03	`this`, Prototypes & Inheritance	12
04	Promises, async/await & Error Propagation	14
05	Garbage Collection & Memory Model	17
06	WeakMap, WeakRef & FinalizationRegistry	19
07	Equality, Coercion & Type Edge Cases	21
08	CommonJS vs ESM — Module Caveats	23
09	Symbols, Proxies & Reflect in Real Systems	25
10	Immutability & Functional Discipline at Scale	27

PART II — NODE.JS IN PRODUCTION

11	libuv, the Thread Pool & Node's Real Event Loop	30
12	Streams & Backpressure	33
13	Cluster, Worker Threads & CPU-Bound Work	36
14	Process Lifecycle & Graceful Shutdown	39
15	Memory Leaks, Heap Snapshots & Profiling	41
16	Error Handling: uncaughtException to Domain Errors	44
17	Security Hardening for Node APIs	46
18	Connection Pooling, Caching & N+1s	49
19	Observability: Logging, Metrics & Tracing	51
20	Scaling, Load Balancing & Resilience Patterns	53

PART III — THE INTERVIEW BANK (90+ QUESTIONS)

A	Conceptual — JavaScript & Node Internals	57
B	Practical / Output-Based — "What does this print?"	62
C	Scenario-Based — System & Production Judgment	67
D	Rapid Fire — Lightning Round	72

How to Use This Book

This is not a JavaScript-from-scratch book. It assumes you already ship Node.js services — this book exists to close the gap between *"it works"* and *"I know exactly why it works, what breaks it in production, and how to defend that in a system design or interview room."*

Every topic in Parts I and II follows the same structure:

- **The concept** — a tight explanation, assuming you know the basics already.
- **A Caveat box** — the thing that bites people in production, not in tutorials.
- **A Production Note** — how senior engineers actually apply this at scale.
- **A Tricky Question box** — a question interviewers love, with the reasoning spelled out.

Part III is a standalone interview bank of 90+ medium-to-hard questions, organized into conceptual, output-prediction ("what does this log?"), scenario-based system-design-flavored questions, and a rapid-fire round for last-minute review. Every answer explains *why*, not just *what* — because in a real interview, the "why" is what's actually being evaluated.

Scope note: since this book is written for backend engineers specifically, it deliberately skips DOM APIs, browser-only quirks, and CSS/rendering topics — every topic here is something you will actually hit inside a Node.js service, a queue worker, or a script running on a server.

PART ONE

JavaScript for the Backend

The language internals that senior engineers reach for when a service behaves strangely at 2am — the event loop, closures, memory, coercion, and modules — not the syntax tutorial you've outgrown.

- 01 The Event Loop, Microtasks & Macrotasks
- 02 Closures, Scope & Memory Leaks
- 03 ``this``, Prototypes & Inheritance
- 04 Promises, `async/await` & Error Propagation
- 05 Garbage Collection & Memory Model
- 06 WeakMap, WeakRef & FinalizationRegistry
- 07 Equality, Coercion & Type Edge Cases
- 08 CommonJS vs ESM – Module Caveats
- 09 Symbols, Proxies & Reflect in Real Systems
- 10 Immutability & Functional Discipline at Scale

01 The Event Loop, Microtasks & Macrotasks

JavaScript is single-threaded, but Node.js is not idle-blocking — the event loop is what reconciles those two facts. Synchronous code runs on the call stack. Once the stack is empty, the loop drains the **microtask queue** completely (Promise callbacks, `queueMicrotask`, `process.nextTick` in Node) before it takes a single task off any macrotask source (timers, I/O callbacks, `setImmediate`).

The interview-critical detail: microtasks fully drain *between every single macrotask*, not once per loop tick. If a microtask enqueues another microtask, it runs before the loop moves on — which is exactly how you can starve I/O with a runaway `Promise` chain.

EXAMPLE — ORDERING

```
console.log('1: sync');

setTimeout(() => console.log('2: macrotask (timer)'), 0);

Promise.resolve().then(() => console.log('3: microtask'));

process.nextTick(() => console.log('4: nextTick'));

console.log('5: sync');

// Output: 1, 5, 4, 3, 2
// nextTick queue drains before the Promise microtask queue in Node
```

⚠ CAVEAT

`process.nextTick()` is **not** part of the standard microtask queue — it's a Node-specific queue that drains even before Promise microtasks, on every phase transition. Recursive `nextTick` calls can starve I/O entirely, which is why Node's docs explicitly warn against unbounded recursive use. This is a real production incident pattern: a retry-on-nextTick bug can make a server appear "hung" while CPU sits at 100%.

✓ PRODUCTION NOTE

When you see event-loop lag in monitoring (e.g. via `perf_hooks.monitorEventLoopDelay()`), the usual cause isn't "too much async" — it's synchronous work hiding inside something that looks async: a large `JSON.parse`, a synchronous regex against attacker-controlled input (ReDoS), or a tight synchronous loop over a big array inside a `.then()`. The loop only starves when the call stack itself stays busy.

+ TRICKY INTERVIEW QUESTION

Q: Will this ever print "done"? What's the output order?

```
async function foo() {  
  console.log('a');  
  await null;  
  console.log('b');  
}  
console.log('start');  
foo();  
console.log('end');
```

A: Output is `start, a, end, b`. Everything in `foo()` before the first `await` runs *synchronously* the moment it's called — an `async function` doesn't defer its own body. The `await` is what schedules the rest of the function as a microtask, so execution returns to the caller (`console.log('end')`) before `b` prints. Candidates who assume the whole function body is deferred get this wrong.

02 Closures, Scope & Memory Leaks

A closure is a function bundled with a reference to its lexical scope — that part every mid-level engineer knows. What separates seniors is knowing that a closure keeps the *entire* variable environment alive, not just the variables it references, in some engine implementations, and that this is a common, hard-to-spot source of memory growth in long-running Node processes.

EXAMPLE — ACCIDENTAL RETENTION

```
function processRequest(largeBuffer) {  
  const summary = summarize(largeBuffer); // small result  
  
  // This inner function only needs `summary`,  
  // but it closes over the whole outer scope —  
  // including `largeBuffer` — keeping it alive.  
  return () => logSummary(summary);  
}  
  
// Fix: null out or scope large data out explicitly  
function processRequestFixed(largeBuffer) {  
  const summary = summarize(largeBuffer);  
  largeBuffer = null; // drop the reference before returning  
  return () => logSummary(summary);  
}
```

⚠ CAVEAT

Closures registered as long-lived listeners (`emitter.on(...)`, cache callbacks, cron jobs) are a classic Node memory leak: each closure keeps its captured scope alive for the lifetime of the listener. If you register a new closure per request and never remove the old listener, you leak one scope per request — this shows up as a slow, linear heap climb, not a spike, which makes it easy to miss in short-lived load tests.

✓ PRODUCTION NOTE

Always pair `emitter.on()` with a matching `removeListener` / `off`, prefer `{ once: true }` for one-shot handlers, and set `emitter.setMaxListeners()` deliberately so Node's default leak warning (10 listeners) actually catches accidental accumulation instead of being silenced globally.

+ TRICKY INTERVIEW QUESTION

Q: Classic loop-and-closure — what does this log, and how do you fix it two different ways?

```
for (var i = 0; i < 3; i++) {  
  setTimeout(() => console.log(i), 0);  
}  
// logs: 3, 3, 3
```

A: `var` is function-scoped, so all three callbacks close over the same `i`, which is `3` by the time the timers fire. Fix 1: swap `var` for `let`, which creates a fresh binding per iteration. Fix 2: wrap the body in an IIFE that captures the current value as a parameter: `((i) => setTimeout(() => console.log(i), 0))(i)`. Interviewers often push further: "what if it's inside a `forEach`?" — same behavior as `let`, since the callback gets its own parameter per call.

03 `this`, Prototypes & Inheritance

`this` is resolved at **call time**, not definition time — determined by how a function is invoked, not where it's written. Four binding rules, in order of precedence: `new` binding, explicit binding (`call` / `apply` / `bind`), implicit binding (called as a method, `obj.method()`), and default binding (bare function call → `undefined` in strict mode, global object otherwise). Arrow functions have no `this` of their own — they lexically inherit it from the enclosing scope, which is why they're the standard fix for callback-context bugs.

EXAMPLE — A REAL PRODUCTION BUG SHAPE

```
class RequestQueue {  
  constructor() { this.pending = []; }  
  
  // Regular method: loses `this` if passed as a bare callback  
  onDrain() { this.pending = []; }  
}  
  
const q = new RequestQueue();  
stream.on('drain', q.onDrain); // BUG: `this` is undefined inside onDrain  
  
// Fix 1: bind explicitly  
stream.on('drain', q.onDrain.bind(q));  
  
// Fix 2: class field arrow function (auto-bound per instance)  
class RequestQueueFixed {  
  pending = [];  
  onDrain = () => { this.pending = []; };  
}
```

⚠ CAVEAT

Class-field arrow functions solve the binding problem but create a *new function per instance* instead of one shared function on the prototype. For a class instantiated per-request in a hot path (e.g. one controller instance per HTTP request), that's extra allocation on every request. For singletons (most Node services), it's a non-issue — know the trade-off, don't cargo-cult it.

✓ PRODUCTION NOTE

Prototype chain lookups are a linear walk up to `Object.prototype`. It's rarely your bottleneck, but deeply nested inheritance hierarchies (common in older ORM-style codebases) do add measurable overhead in property-access-heavy hot loops — favor composition over deep class hierarchies for exactly this reason, independent of the usual design-pattern argument.

+ TRICKY INTERVIEW QUESTION

Q: What does this print, and why?

```
const obj = {
  name: 'service-a',
  greet() { return () => this.name; },
  greetBad: function() { return function() { return this.name; }; }
};
console.log(obj.greet());
console.log(obj.greetBad());
```

A: First call prints `'service-a'` — the arrow function captures `this` lexically from `greet`, where `this` is `obj`. Second call prints `undefined` (or throws in strict mode) — the inner `function` expression gets its own `this`, resolved at call time as a bare invocation, so it defaults instead of inheriting `obj`.

04 Promises, async/await & Error Propagation

A senior engineer's bar for "understands promises" is error handling under concurrency, not chaining `.then()`. Three patterns matter in production: `Promise.all` (fail-fast — one rejection cancels the wait, but does *not* cancel in-flight work), `Promise.allSettled` (always waits for everything, gives you a results array to inspect), and `Promise.race` / `Promise.any` for timeouts and fastest-wins patterns.

EXAMPLE — THE ALL() FOOTGUN

```

async function fanOut(ids) {
  // If ANY request rejects, Promise.all rejects immediately –
  // but the other in-flight requests keep running in the
  // background, unobserved. This is a common source of
  // "phantom" writes: a partial batch already committed
  // server-side even though the caller saw a failure.
  return Promise.all(ids.map(id => fetchAndWrite(id)));
}

// Safer for batch jobs: know exactly what succeeded/failed
async function fanOutSafe(ids) {
  const results = await Promise.allSettled(ids.map(id => fetchAndWrite(id)));
  const failed = results
    .map((r, i) => ({ r, id: ids[i] }))
    .filter(x => x.r.status === 'rejected');
  if (failed.length) logger.warn('partial batch failure', { failed });
  return results;
}

```

⚠ CAVEAT

An `async function` *always* returns a promise, even if you never used `await` inside it — and a thrown synchronous error inside an `async function` becomes a rejected promise, not a thrown exception. If you call an async function without `await` or a `.catch()`, that rejection becomes an **unhandled rejection**, which in modern Node crashes the process by default (`--unhandled-rejections=strict` is the default since Node 15).

✓ PRODUCTION NOTE

"Fire-and-forget" async calls (e.g. a non-critical analytics write inside a request handler) need explicit `.catch()` handlers even when you intentionally don't await them — otherwise a transient failure in a non-critical path can take down the whole process. This is one of the highest-leverage code review checks for Express/Fastify handlers.

+ TRICKY INTERVIEW QUESTION

Q: Does the try/catch here catch the error? Why or why not?

```

async function loadUser(id) {
  try {
    setTimeout(() => { throw new Error('boom'); }, 0);
  } catch (err) {
    console.log('caught:', err.message);
  }
}

```

A: No — the `try/catch` only wraps the synchronous call to `setTimeout`, not the callback that runs later on a separate turn of the event loop. By the time the callback throws, the `try` block has already finished executing and its stack frame is gone. The error becomes an uncaught exception. The fix is to put the `try/catch` (or a `.catch()`) around the actual async operation — e.g. wrap a promise-returning call, not a callback-based timer.

05 Garbage Collection & Memory Model

V8 (Node's engine) uses generational garbage collection. The heap is split into a small **young generation** (Scavenge collector, fast, frequent) and a larger **old generation** (Mark-Sweep-Compact, slower, less frequent). Objects that survive two young-gen collections get "tenured" into the old generation. Most short-lived request-scoped objects (a parsed request body, a temp array) should die in the young generation and never reach old-gen — if they do, at scale, you'll see old-gen GC pauses show up as latency spikes.

⚠ CAVEAT

Old-generation "Mark-Sweep-Compact" GC pauses are **stop-the-world** for the main thread — for a busy Node process with a multi-GB heap, this can mean tens to low-hundreds of milliseconds where nothing else runs, including HTTP responses that were about to be sent. This is why "why do we get periodic p99 latency spikes with no corresponding CPU spike" is so often a GC question, not a code-logic question.

EXAMPLE — DIAGNOSING, NOT FIXING, IN CODE

```
// Run node with these flags to expose GC activity for diagnosis:
// node --trace-gc app.js
// node --max-old-space-size=4096 app.js    (raise the ceiling deliberately)

// Inspect programmatically:
const v8 = require('v8');
console.log(v8.getHeapStatistics());
// { total_heap_size, used_heap_size, heap_size_limit, ... }
```

✓ PRODUCTION NOTE

The default old-space limit (~2GB on 64-bit Node prior to explicit sizing, higher on recent versions but still finite) is a real production ceiling — a service that "just needs more memory" without raising `--max-old-space-size` will crash with **JS heap out of memory** instead of relying on the container's memory limit. Always size this deliberately relative to your container's memory limit, leaving headroom for non-heap memory (buffers, native addons).

+ TRICKY INTERVIEW QUESTION

Q: Your service's RSS memory climbs steadily under constant load and never plateaus, but heap snapshots show the JS heap is stable. What's happening, and where do you look?

A: This points away from the JS heap entirely — toward off-heap growth: Buffers (which allocate outside V8's heap), native addon memory, or unbounded growth in things GC can't see, like an ever-growing native `Map` inside a C++ addon, unclosed file descriptors, or accumulating event listeners on native objects. The diagnostic move is comparing `process.memoryUsage()`'s `heapUsed` vs `rss` vs `external` fields over time — a stable heap with climbing `external` / `rss` strongly implicates Buffers or native memory, not JS-level leaks.

06 WeakMap, WeakRef & FinalizationRegistry

`WeakMap` and `WeakSet` hold their keys **weakly** — if nothing else references a key, it's eligible for garbage collection even while it's still a key in the map, and the entry disappears with it. This makes them the correct primitive for metadata that should

live and die with an object, without the map itself becoming a leak.

EXAMPLE — REQUEST-SCOPED METADATA CACHE

```
// Regular Map: leaks forever if you forget to delete()
const metaCache = new Map();
function attachMeta(req, data) { metaCache.set(req, data); }
// req objects pile up in metaCache even after the request ends

// WeakMap: entry is collected automatically once `req` is unreachable
const metaCacheWeak = new WeakMap();
function attachMetaSafe(req, data) { metaCacheWeak.set(req, data); }
```

⚠ CAVEAT

`WeakMap` / `WeakSet` are **not iterable** and have no `.size` — that's not an oversight, it's fundamental: since entries can disappear at any moment via GC, exposing a count or iterator would be non-deterministic. If you need to iterate cache entries for debugging or eviction logic, a `WeakMap` is the wrong tool; you want a regular `Map` with an explicit TTL/LRU eviction strategy instead.

✓ PRODUCTION NOTE

`WeakRef` and `FinalizationRegistry` (ES2021) are rarely something you reach for directly in application code — they're mostly infrastructure for library authors building caches or resource managers. Node's own docs and V8 team explicitly caution against relying on `FinalizationRegistry` callback timing for correctness (cleanup timing is not guaranteed or prompt) — never use it to release a scarce resource like a file handle or DB connection; use explicit `try/finally` or `using` declarations for that.

+ TRICKY INTERVIEW QUESTION

Q: Why can't you do `[...weakMap.keys()]` or `JSON.stringify(weakMap)` ?

A: Both require enumerating entries, and `WeakMap` deliberately exposes no enumeration API. This isn't a missing feature — it's the mechanism that makes the "weak" guarantee possible. If you could list all keys, the engine would need to guarantee those keys stay alive and stable during iteration, which directly conflicts with letting GC reclaim them silently. A strong interview answer connects the *design constraint* to the *missing API*, not just "it's not supported."

07 Equality, Coercion & Type Edge Cases

`==` performs type coercion before comparing; `===` never coerces. The senior-level nuance isn't "always use `===`" (true, but table stakes) — it's knowing the specific coercion rules well enough to spot them in code review, and understanding where implicit coercion causes real production bugs: query-string parsing, JSON round-trips, and database driver type mapping.

EXAMPLE — WHERE THIS ACTUALLY BITES

```
// req.query values are ALWAYS strings, even for numeric-looking params
// GET /items?limit=10
req.query.limit === 10      // false – it's the string '10'
req.query.limit == 10       // true – coerced, but masks a type bug elsewhere
Number(req.query.limit) === 10 // true – explicit, correct

// Classic gotchas worth having memorized:
[] == false      // true ([] -> '' -> 0, false -> 0)
[] == ![]        // true (![] is false, then as above)
null == undefined // true (special-cased, loosely equal ONLY to each other)
null === undefined // false
NaN === NaN      // false – NaN is never equal to itself
Object.is(NaN, NaN) // true – Object.is fixes the NaN and -0/+0 edge cases
0.1 + 0.2 === 0.3 // false – IEEE-754 float precision
```

⚠ CAVEAT

`Array.prototype.includes()` uses the `SameValueZero` algorithm, so `[NaN].includes(NaN)` is `true` — but `[NaN].indexOf(NaN)` is `-1`, because `indexOf` uses strict equality. This exact discrepancy has shipped real bugs when engineers swap one for the other assuming identical semantics.

✓ PRODUCTION NOTE

Never compare floating-point money or measurement values with `===`. Use integer minor-units (store cents, not dollars) or a fixed-precision/decimal library for financial calculations — this is a recurring root cause in payment-system bugs, not a theoretical concern.

+ TRICKY INTERVIEW QUESTION

Q: Sort this array with the default `.sort()` — what do you get, and why?

```
[10, 1, 21, 2].sort();
```

A: `[1, 10, 2, 21]`. The default comparator converts every element to a **string** and sorts lexicographically — so `'10'` sorts before `'2'`. This is a near-universal gotcha; the fix is always an explicit comparator: `arr.sort((a, b) => a - b)`. It's a favorite interview question precisely because it's easy to miss in real code — an array that happens to already contain only single-digit numbers will hide the bug until production data introduces a two-digit value.

08 CommonJS vs ESM — Module Caveats

CommonJS (`require`) is synchronous, resolves at call time, and exports a **live-ish but mutable** object — mutating `module.exports` after another file has already required it doesn't retroactively update their copy of the reference (though mutating properties *on* the exported object does propagate, since both hold the same object reference). ESM (`import` / `export`) is asynchronous under the hood, statically analyzable (imports are hoisted and resolved before execution), and exports **live bindings** — a change to an exported variable in the source module *is* visible to importers, automatically.

EXAMPLE — THE LIVE-BINDING DIFFERENCE

```
// counter.mjs (ESM)
export let count = 0;
export function increment() { count++; }

// main.mjs
import { count, increment } from './counter.mjs';
console.log(count); // 0
increment();
console.log(count); // 1 — the binding is LIVE, this really updates

// The CommonJS equivalent would NOT do this — destructuring
// `const { count } = require('./counter')` copies the value once.
```

⚠ CAVEAT

Circular `require()` s in CommonJS return a **partial, possibly-incomplete** exports object (whatever was defined before the circular point) rather than throwing — this fails silently and is a real source of "why is this function undefined" bugs in large codebases with tangled module graphs. ESM handles circular imports more gracefully via live bindings and hoisted declarations, but can still throw a `ReferenceError` if you access a binding before its declaration actually executes (the "temporal dead zone" of module init).

✓ PRODUCTION NOTE

Mixing the two in one codebase is a common real-world pain point: an ESM project can `import` a CJS package (Node synthesizes a default export), but a CJS file cannot `require()` a pure-ESM package synchronously — you need a dynamic `import()`, which is async. This regularly breaks when upgrading a dependency that switches to `"type": "module"` — know this failure mode before it surprises you in a dependency bump PR.

+ TRICKY INTERVIEW QUESTION

Q: Two files require each other (A requires B, B requires A). What happens, concretely?

A: Node detects the cycle and breaks it by returning whatever `module.exports` looks like *at the point of the circular require* — not the final version. If B is required by A while B is still in the middle of executing (and B in turn requires A), B receives A's exports object as it existed at that moment, which may be empty or partial (e.g. `{}` if A hadn't assigned anything to `module.exports` yet). The fix in practice: restructure to remove the cycle, or move the cross-reference inside a function body so it's resolved lazily at call time instead of at module-load time.

09 Symbols, Proxies & Reflect in Real Systems

`Symbol` creates guaranteed-unique property keys — used for defining "hidden" object protocol hooks (`Symbol.iterator`, `Symbol.asyncIterator`) without risking collision with a user's string keys. `Proxy` wraps an object and lets you intercept fundamental operations (get, set, has, deleteProperty, etc.) — the real-world use is building things like ORMs, validation layers, and reactive state systems, not everyday application code.

EXAMPLE — A VALIDATING CONFIG PROXY

```
function createValidatedConfig(schema) {
  return new Proxy({}, {
    set(target, key, value) {
      if (!schema[key]) throw new Error(`Unknown config key: ${key}`);
      if (typeof value !== schema[key])
        throw new TypeError(`${key} must be ${schema[key]}`);
      return Reflect.set(target, key, value);
    }
  });
}

const config = createValidatedConfig({ port: 'number', host: 'string' });
config.port = '8080'; // throws: TypeError – must be 'number'
```

⚠ CAVEAT

Proxies add real overhead to every intercepted operation — they are not free indirection. Wrapping a hot-path object (e.g. something touched per-iteration in a tight loop processing thousands of records) in a `Proxy` for validation can measurably regress performance. Reserve them for boundaries (config objects, API surfaces, dev-mode instrumentation), not inner-loop data structures.

✓ PRODUCTION NOTE

`Reflect` methods mirror the default behavior a `Proxy` trap would otherwise have, and using `Reflect.set` / `Reflect.get` inside your trap (instead of directly mutating `target`) correctly preserves things like inherited setters and receiver semantics — using `Reflect` inside Proxy traps is the idiomatic, correctness-preserving pattern, not an optional style choice.

+ TRICKY INTERVIEW QUESTION

Q: Why use `Symbol('id')` instead of a string key like `'id'` for private-ish metadata on a shared object?

A: Two reasons interviewers look for: (1) uniqueness — `Symbol('id') !== Symbol('id')`, even with the same description, so two independent libraries attaching metadata to the same object can't accidentally collide or overwrite each other's key. (2) Symbols are excluded from `Object.keys()`, `for...inJSON.stringify()` by default, which makes them a reasonable (not cryptographically secure) way to attach "hidden" metadata that won't leak through normal serialization or enumeration. It is not true privacy — `Object.getPrototypeOfSymbols()` can still find them.

10 Immutability & Functional Discipline at Scale

`Object.freeze()` is **shallow** — it prevents reassigning top-level properties but does nothing to nested objects. This single fact is responsible for a large share of "I froze it but it still mutated" bugs. True deep immutability requires recursively freezing, or reaching for a structural-sharing library (Immer, Immutable.js) for anything beyond trivial nesting.

EXAMPLE — THE SHALLOW-FREEZE TRAP


```
const config = Object.freeze({
  server: { port: 8080 },
  name: 'api'
});

config.name = 'changed'; // silently no-op in non-strict, throws in strict mode
config.server.port = 9090; // WORKS – nested object is NOT frozen
console.log(config.server.port); // 9090

// Deep freeze utility, recursively:
function deepFreeze(obj) {
  Object.getOwnPropertyNames(obj).forEach(key => {
    const val = obj[key];
    if (val && typeof val === 'object') deepFreeze(val);
  });
  return Object.freeze(obj);
}
```

⚠ CAVEAT

In non-strict mode, assigning to a frozen property fails **silently** — no error, no warning, the assignment just doesn't happen. This is a dangerous default for catching bugs early; always run Node with `'use strict'` semantics (ES modules are strict by default; CJS files should opt in explicitly) so violations throw a `TypeError` you'll actually notice in testing.

✓ PRODUCTION NOTE

Immutable update patterns (spreading, `.map` / `.filter` instead of in-place mutation) matter most where shared references cause action-at-a-distance bugs — Redux-style state, cache objects handed to multiple consumers, config objects passed through middleware chains. For purely local, hot-path data (e.g. accumulating results in a tight loop), forcing immutability adds real allocation overhead for no correctness benefit — know when the discipline is buying you safety versus just costing you GC pressure.

+ TRICKY INTERVIEW QUESTION

Q: Given a deeply nested state object in a Node service shared across async handlers, what's the actual production risk of mutating it directly, and how would you catch it in code review?

A: The core risk is a race-condition-like bug without real threading: handler A reads the shared object, awaits an I/O call, and while it's suspended, handler B mutates the same object. When A resumes, it's now operating on state it didn't expect — a class of bug that's hard to reproduce because it depends on request timing/interleaving. In review, the tell is any shared, mutable object (a cache entry, a singleton config, a passed-by-reference request context) being written to inside a function that also does an `await` before or after the write. The fix is either treating shared state as immutable (copy-on-write) or narrowing what's shared to begin with.

PART TWO

Node.js in Production

Everything above the language itself: how Node actually schedules I/O, where backpressure and memory leaks come from, and the operational patterns that keep a service alive under real traffic.

- 11 libuv, the Thread Pool & Node's Real Event Loop
- 12 Streams & Backpressure
- 13 Cluster, Worker Threads & CPU-Bound Work
- 14 Process Lifecycle & Graceful Shutdown
- 15 Memory Leaks, Heap Snapshots & Profiling
- 16 Error Handling: uncaughtException to Domain Errors
- 17 Security Hardening for Node APIs
- 18 Connection Pooling, Caching & N+1s
- 19 Observability: Logging, Metrics & Tracing
- 20 Scaling, Load Balancing & Resilience Patterns

11 libuv, the Thread Pool & Node's Real Event Loop

Node's event loop runs in distinct **phases**, each with its own callback queue, cycled in order: *timers* (setTimeout/setInterval), *pending callbacks* (deferred I/O callbacks), *poll* (retrieve new I/O events, execute I/O callbacks — this is where most work happens and where the loop can block waiting for I/O), *check* (`setImmediate`), and *close callbacks*. Between every phase transition, Node drains `process.nextTick` and then the Promise microtask queue.

Underneath, **libuv** (Node's C library) provides a fixed-size thread pool (default 4 threads, tunable via `UV_THREADPOOL_SIZE`) that handles operations the OS can't do async natively: most file-system calls, DNS lookups (`dns.lookup` , not `dns.resolve`), and some crypto functions (`pbkdf2` , `scrypt`). Network I/O, by contrast, uses the OS's native async mechanisms (epoll/kqueue/IOCP) and doesn't touch the thread pool at all.

EXAMPLE — SETIMMEDIATE VS SETTIMEOUT(0) ORDERING

```
// Inside the main module (top-level), order is NOT guaranteed —
// it depends on process startup timing:
setTimeout(() => console.log('timeout'), 0);
setImmediate(() => console.log('immediate'));

// But inside an I/O callback, setImmediate ALWAYS wins —
// because the poll phase hands off directly to the check phase:
fs.readFile(__filename, () => {
  setTimeout(() => console.log('timeout'), 0);
  setImmediate(() => console.log('immediate')); // always first here
});
```

⚠ CAVEAT

File I/O in Node is not "async" in the same sense as network I/O — it's synchronous work farmed out to a small, shared thread pool. Four concurrent file reads will saturate the default pool; a fifth waits, even though your JS code looks fully non-blocking. Under load, this is a real, easy-to-miss bottleneck for services that do heavy file or DNS-lookup work per request.

✓ PRODUCTION NOTE

Raise `UV_THREADPOOL_SIZE` (as an environment variable, set *before* Node starts — it cannot be changed at runtime) if your service is genuinely thread-pool-bound: heavy `bcrypt` / `crypto.pbkdf2` usage, lots of file I/O, or DNS-heavy fan-out. Prefer `dns.resolve()` (uses c-ares, not the thread pool) over `dns.lookup()` when you don't need to match the OS's own resolution behavior (e.g. `/etc/hosts` entries).

+ TRICKY INTERVIEW QUESTION

Q: Your Node API does synchronous bcrypt hashing on every login request. Under load-testing, p99 latency for unrelated, simple GET endpoints spikes too. Why?

A: `bcrypt` (and Node's built-in `crypto.pbkdf2` / `scrypt`) run on the libuv thread pool, not the main JS thread — so they don't block the event loop directly. But the pool is small (default 4 threads) and shared across *all* thread-pool-dependent work in the process, including any file reads those "unrelated" GET endpoints might do (serving a static asset, reading a config file, etc). Heavy concurrent bcrypt calls exhaust the pool, so unrelated file-I/O-dependent requests queue up waiting for a free thread — even though their own code has nothing to do with authentication. The fix: move password hashing to a worker thread pool or a separate microservice, and/or raise `UV_THREADPOOL_SIZE` deliberately.

12 Streams & Backpressure

Streams let you process data in chunks instead of loading it all into memory — essential for anything involving large files, uploads, or proxying between services. **Backpressure** is the mechanism that keeps a fast producer from overwhelming a slow consumer: when you call `writable.write(chunk)` and it returns `false` , the internal buffer has exceeded `highWaterMark` , and you must stop writing until the stream emits `'drain'` .

EXAMPLE — THE NAIVE VS CORRECT PATTERN

```
// WRONG: ignores backpressure entirely – buffers unboundedly
// in memory if the destination (disk, network) is slower than the source
readable.on('data', chunk => writable.write(chunk));

// CORRECT: pipe() handles backpressure automatically
readable.pipe(writable);

// CORRECT (manual control, e.g. when you need to transform first):
readable.on('data', chunk => {
  const ok = writable.write(chunk);
  if (!ok) {
    readable.pause();
    writable.once('drain', () => readable.resume());
  }
});

// BEST for chained transforms: pipeline() – also handles cleanup
// and error propagation across the whole chain automatically
const { pipeline } = require('stream/promises');
await pipeline(readable, transformStream, writable);
```

⚠ CAVEAT

Manually chaining `.pipe()` calls does **not** propagate errors between streams — an error on a middle transform stream does not automatically destroy the source or destination streams, which can leave file handles and sockets open indefinitely. This is one of the most common causes of file-descriptor leaks in Node services that process uploads. `pipeline()` (from `stream/promises` or the callback version) exists specifically to fix this — it always cleans up every stream in the chain on error.

✓ PRODUCTION NOTE

When proxying large request/response bodies (file uploads, video, large API payloads) between services, always stream — never buffer the full body into memory with something like `concat-stream` or manual chunk accumulation before forwarding it. A service accepting concurrent large uploads will OOM under load if every request fully buffers before forwarding, even though the same code passes every functional test with small payloads.

+ TRICKY INTERVIEW QUESTION

Q: You're piping a 10GB file to a slow network destination. Memory usage stays flat throughout. Explain why, in terms of what's actually happening at the OS and stream level.

A: This is backpressure working correctly. `.pipe()` reads a chunk, writes it, and checks the return value of `write()` — when the destination's internal buffer fills up (write returns `false`, meaning the underlying socket's kernel send buffer is also likely under pressure), `pipe()` automatically calls `.pause()` on the readable side, halting further reads from disk until a `'drain'` event confirms the writable side has caught up. Memory stays flat because only a bounded number of chunks (governed by `highWaterMark`, default 64KB per stream) are ever in flight at once — the file is never loaded into memory in its entirety.

13 Cluster, Worker Threads & CPU-Bound Work

Node is single-threaded for JS execution — a CPU-intensive synchronous operation (image resizing, heavy JSON transforms, cryptographic work not on the thread pool, complex regex) blocks the entire event loop, stalling every other request the process is handling. Two distinct tools solve two distinct problems: **cluster** (or a process manager like PM2) forks multiple full Node processes to use multiple CPU cores for handling more concurrent I/O-bound requests; **worker_threads** runs actual JS in a separate thread within the same process, for offloading genuinely CPU-bound work without blocking the main event loop.

EXAMPLE — OFFLOADING CPU WORK CORRECTLY

```
// main.js
const { Worker } = require('worker_threads');

function runHeavyTask(data) {
  return new Promise((resolve, reject) => {
    const worker = new Worker('./heavy-task.js', { workerData: data });
    worker.on('message', resolve);
    worker.on('error', reject);
  });
}

// The main event loop stays free to handle other requests
// while heavy-task.js does synchronous CPU work on its own thread.
```

⚠ CAVEAT

`cluster` gives you N processes with N **separate memory spaces and separate event loops** — an in-memory cache, a WebSocket connection registry, or an in-process rate limiter built with a plain JS `Map` will silently diverge across workers, since each worker has its own copy. This is a very common source of "it works locally but breaks with more than one instance" bugs — the fix is externalizing shared state to Redis or another out-of-process store, not adding more in-memory data structures.

✓ PRODUCTION NOTE

Worker threads communicate via structured cloning by default (message passing, copies data) — for large payloads this copy cost matters. `SharedArrayBuffer` allows true zero-copy shared memory between threads for numeric/binary data, but requires manual synchronization (`Atomics`) since you now have real shared mutable state, with real race conditions, unlike anywhere else in typical Node code.

✦ TRICKY INTERVIEW QUESTION

Q: Your team wants to add an in-memory rate limiter (a plain `Map` tracking request counts per IP) to a service running 4 instances behind a load balancer via `cluster`. What's wrong with this design, concretely?

A: Each of the 4 clustered processes has its own independent `Map` — the load balancer distributes requests across workers (commonly round-robin), so a client making 100 requests might land roughly 25 on each worker. Each worker's rate limiter only sees its own 25 requests and never trips, even though the client's total request rate against the service is 4x whatever the limiter's threshold is. The fix is a shared, out-of-process store for the counters — Redis with atomic `INCR` and TTL is the standard answer — so all workers see and enforce against the same count.

14 Process Lifecycle & Graceful Shutdown

In containerized/orchestrated environments (Kubernetes, ECS), your process receives `SIGTERM` before being forcibly killed with `SIGKILL` after a grace period (Kubernetes defaults to 30 seconds). A service that doesn't handle `SIGTERM` deliberately gets killed mid-request on every deploy and every scale-down event — this is a direct, common cause of dropped requests during rolling deploys.

EXAMPLE — A REAL GRACEFUL SHUTDOWN SEQUENCE

```

const server = app.listen(port);
let isShuttingDown = false;

process.on('SIGTERM', async () => {
  if (isShuttingDown) return;
  isShuttingDown = true;
  logger.info('SIGTERM received, draining...');

  // 1. Stop accepting NEW connections, let in-flight ones finish
  server.close(() => logger.info('HTTP server closed'));

  // 2. Fail readiness probe immediately so the LB stops routing here
  isReady = false;

  // 3. Close DB pools, flush queues, etc – with a hard timeout
  await Promise.race([
    Promise.all([dbPool.end(), queue.close()]),
    new Promise(r => setTimeout(r, 10_000))
  ]);

  process.exit(0);
});

```

⚠ CAVEAT

Kubernetes sends `SIGTERM` to your process and *simultaneously* starts removing the pod from Service endpoints — but these two things race, and your readiness probe (if you have one) is the actual signal that stops new traffic. If your app doesn't fail its readiness check on shutdown start, a handful of new requests can still land on a pod that's already draining, right up until the endpoint removal propagates — always flip readiness to "not ready" as the very first action in your shutdown handler.

✓ PRODUCTION NOTE

Always set a hard timeout that force-exits (or lets Kubernetes's `SIGKILL` do it) if graceful cleanup hangs — a single stuck DB connection during shutdown, without a timeout guard, can turn a clean 5-second drain into the full grace-period wait on every single deploy, materially slowing down rollout speed at scale.

+ TRICKY INTERVIEW QUESTION

Q: Rolling deploys cause a brief spike in 502 errors, even though your app handles `SIGTERM` correctly and drains in-flight requests. What else could be wrong?

A: A correct in-app shutdown handler isn't sufficient by itself — the common gaps are: (1) no `preStop` hook / readiness-flip delay, so the load balancer hasn't yet noticed the pod is going away before traffic stops being routed to it; (2) the new pod's readiness probe passes before the app has actually finished warming up (e.g. before DB connection pools are established), so traffic hits it before it's really ready; (3) keep-alive connections on the client/proxy side holding onto a socket to the old pod after the server stops accepting new requests on it. Interviewers are testing whether you think about the deploy as a distributed-systems handoff between load balancer, orchestrator, and app — not just an in-process signal handler.

15 Memory Leaks, Heap Snapshots & Profiling

The standard workflow for diagnosing a suspected Node memory leak: capture two or more heap snapshots under load, separated by time, using Chrome DevTools' "Comparison" view (via `--inspect`) or the `heapdump` / `v8.writeHeapSnapshot()` API — then look at what's growing in **retained size** between snapshots, not just what exists. A single snapshot tells you what's alive; a comparison tells you what's *accumulating*, which is what actually matters for a leak.

EXAMPLE — CAPTURING A SNAPSHOT IN PRODUCTION-SAFE WAY

```
const v8 = require('v8');
const fs = require('fs');

// Trigger on demand (e.g. via an internal admin endpoint,
// NEVER exposed publicly – a snapshot can contain sensitive data)
function captureSnapshot() {
  const snapshotStream = v8.getHeapSnapshot();
  const file = fs.createWriteStream(`/tmp/heap-${Date.now()}.heapsnapshot`);
  snapshotStream.pipe(file);
}

// Load the .heapsnapshot files into Chrome DevTools → Memory tab
// → use "Comparison" between two snapshots taken minutes apart.
```

⚠ CAVEAT

Taking a heap snapshot forces a full garbage collection and **pauses the process** — on a large heap (multi-GB), this pause can be seconds long. Never wire automatic periodic snapshotting into a production hot path; gate it behind an explicit, access-controlled trigger and expect a real, user-visible latency hit for that instance when you use it.

✓ PRODUCTION NOTE

The most common real-world Node "leaks" aren't exotic — they're unbounded caches (a `Map` used as a cache with no eviction policy), forgotten `setInterval` timers that are never `clearInterval`'d, event listeners accumulating on a shared `EventEmitter`, and closures capturing large objects (Topic 02). Before reaching for heap snapshots, grep the codebase for exactly those four patterns — it resolves the leak faster than profiling in a large percentage of real cases.

+ TRICKY INTERVIEW QUESTION

Q: `process.memoryUsage().heapUsed` climbs steadily over 6 hours, then drops sharply, then climbs again — a sawtooth pattern. Is this a leak?

A: Not necessarily — a sawtooth is actually the *expected, healthy* shape of a GC'd heap under steady load: usage climbs as objects accumulate in the young/old generation, then a major GC cycle reclaims unreachable objects and usage drops. A real leak looks different: the *bottom* of each sawtooth trends upward over time — each GC cycle reclaims less than the last, because more and more objects are being kept alive by a genuine reference that GC correctly refuses to collect. The interview-correct answer is to look at the floor of the sawtooth pattern over a long window, not the peaks, and not a single climb-then-drop in isolation.

16 Error Handling: `uncaughtException` to Domain Errors

`process.on('uncaughtException')` and `process.on('unhandledRejection')` are **last-resort safety nets, not error-handling strategy**. Node's own documentation is explicit: after an uncaught exception, the process is in an unknown state — continuing to run risks memory leaks, resource leaks, and undefined behavior. The correct pattern is to log everything you can, attempt a clean shutdown, and let the process manager (Kubernetes, PM2, systemd) restart it.

EXAMPLE — THE CORRECT LAST-RESORT HANDLER

```
process.on('uncaughtException', (err, origin) => {
  logger.fatal({ err, origin }, 'uncaught exception - shutting down');
  // Give logs a moment to flush, then exit non-zero so the
  // orchestrator knows this was a crash, not a clean stop.
  setTimeout(() => process.exit(1), 1000).unref();
});

process.on('unhandledRejection', (reason, promise) => {
  logger.fatal({ reason }, 'unhandled rejection - shutting down');
  process.exit(1);
});
```

⚠ CAVEAT

Distinguish **operational errors** (expected failure modes: a downstream API timeout, invalid user input, a DB connection drop — these should be caught, logged, and handled gracefully with a proper HTTP response) from **programmer errors** (a bug: calling a method on `undefined`, a broken invariant — these indicate the program is in a state its author never anticipated, and should generally crash and restart rather than be "handled" and papered over). Treating every error the same way — catching everything, always continuing — hides real bugs and lets corrupted state limp along in production.

✓ PRODUCTION NOTE

Use custom error classes (extending `Error`) with a stable, typed shape — e.g. an `isOperational: true` flag, an HTTP status code, an error code string — so centralized error-handling middleware can make consistent decisions without string-matching error messages, which breaks silently the moment someone edits a message for clarity.

+ TRICKY INTERVIEW QUESTION

Q: Should you catch an error, log it, and continue serving requests — or let the process crash and restart? Give the actual decision rule.

A: The rule is about state integrity, not error severity: if the error occurred in a context where you can be confident the rest of the process's in-memory state is still valid and uncorrupted (e.g. a well-scoped try/catch around a single request handler, where the error can't have left shared state half-mutated), catching and continuing is correct — that's exactly what request-scoped error handling middleware is for. If the error surfaces via `uncaughtException` / `unhandledRejection` — meaning it escaped every scoped handler — you no longer have that confidence, because you don't know what shared state the failing code path may have partially mutated before throwing. That's the case for logging, shutting down cleanly, and letting the process manager restart with fresh state.

17 Security Hardening for Node APIs

A senior-level security checklist for a Node API goes well beyond "use helmet": validate and sanitize all input at the boundary (never trust `req.body` / `req.query` shapes — a schema validator like Zod or Joi, not ad-hoc checks), use parameterized queries or an ORM's query builder exclusively (never string-concatenate SQL or Mongo query objects built from raw input), rate-limit per-identity (not just per-IP, which is trivially defeated behind shared NATs or proxies), and keep secrets out of source control and environment dumps alike.

EXAMPLE — A REAL NOSQL INJECTION VECTOR, OFTEN MISSED

```
// VULNERABLE: MongoDB query operators can be injected via JSON body
// POST body: { "username": "admin", "password": { "$ne": null } }
const user = await User.findOne({
  username: req.body.username,
  password: req.body.password // { $ne: null } matches ANY password
});
// This bypasses auth entirely — no SQL involved, "SQL injection"
// training alone doesn't cover it.

// FIX: validate shape/type BEFORE it ever reaches the query layer
const schema = z.object({
  username: z.string(),
  password: z.string() // rejects the object payload outright
});
const { username, password } = schema.parse(req.body);
```

⚠ CAVEAT

`helmet` and similar middleware set reasonable default headers, but they are not a substitute for input validation — they mitigate a specific, narrow class of browser-side attacks (clickjacking, some XSS vectors, MIME sniffing). A NoSQL injection, a broken object-level authorization check (fetching `/orders/:id` without verifying the order belongs to the requesting user), or a leaked JWT secret are untouched by security headers entirely — don't let "we added helmet" create false confidence in a security review.

✓ PRODUCTION NOTE

Rate limiting by IP alone is close to useless against a distributed or authenticated attacker — always layer identity-based limits (per API key, per user ID, per session) on top, and treat IP-based limiting as only a coarse first line of defense against unauthenticated volumetric abuse.

+ TRICKY INTERVIEW QUESTION

Q: An endpoint `GET /api/invoices/:id` checks `req.user` is authenticated (a valid JWT) but doesn't check anything else. What's the vulnerability class, and how do you fix it?

A: This is **broken object-level authorization** (IDOR — insecure direct object reference), consistently one of the top real-world API vulnerability classes in practice. Any authenticated user can enumerate `:id` values and read invoices belonging to other users/tenants, because the check confirms *who you are* but never confirms *you're allowed to see this specific resource*. The fix is enforcing ownership/tenancy at the query level — e.g. `Invoice.findOne({ _id: id, ownerId: req.user.id })` rather than `Invoice.findById(id)` followed by a separate, easily-forgotten permission check — making the unauthorized case structurally return "not found" rather than relying on an if-statement elsewhere in the handler.

18 Connection Pooling, Caching & N+1s

Every DB driver's connection pool has a finite size — and a pool that's too small serializes requests waiting for a free connection, while a pool that's too large can overwhelm the database server itself (each DB connection has real memory/resource cost on the DB side too). The right size is a function of your DB's actual connection ceiling divided across all service instances, not a number picked because it "felt right."

EXAMPLE — THE N+1 QUERY PATTERN, AND THE FIX

```
// N+1: one query for orders, then ONE MORE query PER ORDER
const orders = await Order.find({ userId });
for (const order of orders) {
  order.items = await OrderItem.find({ orderId: order.id }); // N queries
}

// FIX: batch it into a single query, then group in memory
const orders = await Order.find({ userId });
const orderIds = orders.map(o => o.id);
const allItems = await OrderItem.find({ orderId: { $in: orderIds } });
const itemsByOrder = groupBy(allItems, 'orderId');
orders.forEach(o => o.items = itemsByOrder[o.id] ?? []);
```

⚠ CAVEAT

N+1 queries are invisible in local development (tiny dataset, one user, zero latency to a local DB) and only show up as a real problem under production data volume and network latency to the DB — which is exactly why they survive code review so often. Query-count assertions in integration tests (asserting a request triggers no more than N queries) catch this class of bug far more reliably than eyeballing code.

✓ PRODUCTION NOTE

Cache invalidation is the genuinely hard part, not the caching itself. Prefer short TTLs with acceptable staleness over complex manual invalidation logic wherever the product tolerates it — a 30-second-stale cache with zero invalidation bugs usually beats a "perfectly fresh" cache with an invalidation path someone will eventually forget to call after adding a new write path.

+ TRICKY INTERVIEW QUESTION

Q: Under a sudden traffic spike, a popular cache key expires and hundreds of concurrent requests all miss the cache at once, hammering the database simultaneously. What's this called, and how do you prevent it?

A: This is the **"thundering herd"** (or "cache stampede") problem. The standard fixes: (1) request coalescing — the first request to miss the cache acquires a lock/flag and fetches from the DB while concurrent requests for the same key wait on that in-flight fetch instead of independently hitting the DB; (2) probabilistic early expiration — refresh the cache slightly before actual TTL expiry, with jitter, so keys don't all expire in lockstep; (3) stale-while-revalidate — serve the slightly-stale cached value immediately while refreshing in the background, rather than blocking on a fresh fetch at all. Interviewers are checking whether you know cache misses can correlate under load in a way that defeats the purpose of caching entirely.

19 Observability: Logging, Metrics & Tracing

The three pillars serve different questions: **logs** answer "what exactly happened in this one request" (structured, not `console.log`-formatted strings — JSON logs with consistent fields are what let you actually query them at scale); **metrics** answer "what's the aggregate system behavior over time" (request rate, error rate, latency percentiles — p50/p95/p99, never just average, since averages hide tail latency that individual users actually feel); **traces** answer "where did time actually go across this request's journey through multiple services."

EXAMPLE — STRUCTURED LOGGING WITH CORRELATION

```
// Every log line should carry a request/trace ID so you can
// reconstruct one request's full story across services and logs.
app.use((req, res, next) => {
  req.id = req.headers['x-request-id'] ?? crypto.randomUUID();
  req.log = logger.child({ requestId: req.id }); // scoped child logger
  next();
});

app.get('/orders/:id', async (req, res) => {
  req.log.info({ orderId: req.params.id }, 'fetching order');
  // ...every log line in this request carries the same requestId
});
```

⚠ CAVEAT

Logging the full request/response body "for debugging" is a recurring compliance and security incident source — request bodies routinely contain passwords, tokens, PII, and payment data. Always define an explicit allowlist/redaction policy for logged fields before shipping a logging change, rather than logging everything and hoping to remember to redact later.

✓ PRODUCTION NOTE

Average latency is close to a vanity metric for user-facing systems — a service with 50ms average and a 4-second p99 is failing a meaningful fraction of real users while looking perfectly healthy on a dashboard that only shows the mean. Always alert on p95/p99, and always break metrics down by route/endpoint, not just as a single aggregate across the whole service.

+ TRICKY INTERVIEW QUESTION

Q: Your dashboard shows average latency at 80ms, well within SLA — but you're getting user complaints about slow requests. How do you investigate, and what's the likely root cause class?

A: Pull up the p99 (and ideally p99.9) latency, not the average — averages are dominated by the high-volume fast path and can completely hide a smaller but real population of slow requests. The likely root cause class is a specific slow path affecting a subset of requests: a particular query missing an index that only manifests for accounts with large datasets, a specific downstream dependency with intermittent slowness, or GC pauses correlating with specific request patterns. The concrete next step is segmenting latency by route, by customer/tenant size, and by time — a tail-latency problem is almost always concentrated in a specific dimension, not evenly distributed, and finding that dimension is the actual debugging work.

20 Scaling, Load Balancing & Resilience Patterns

Horizontal scaling (more instances) is the default answer for Node specifically, because a single process can only use one CPU core for JS execution — vertical scaling (a bigger machine) mostly just wastes the extra cores unless you're also running `cluster` /worker threads to use them. Resilience at scale means assuming any downstream dependency can and will fail, and designing so one failure doesn't cascade into a total outage.

EXAMPLE – CIRCUIT BREAKER PATTERN, CONCEPTUALLY

```
// Without a circuit breaker: every request to a dead downstream
// service waits out the FULL timeout before failing – under load,
// this exhausts your own connection pool / event loop capacity
// waiting on a dependency that isn't coming back soon.

const breaker = new CircuitBreaker(callPaymentService, {
  timeout: 3000,           // fail fast per-call
  errorThresholdPercentage: 50,
  resetTimeout: 30000      // after opening, try again after 30s
});
breaker.fallback(() => ({ status: 'queued_for_retry' }));

// After enough failures, the breaker "opens": subsequent calls
// fail IMMEDIATELY without even attempting the network call,
// giving the downstream service room to recover instead of being
// hammered by retries from every failing request.
```

⚠ CAVEAT

Naive retry logic ("just retry 3 times on failure") without exponential backoff and jitter is a well-known way to *cause* an outage rather than survive one — if a downstream service degrades and every client retries immediately and simultaneously, the retry traffic itself can be the thing that prevents recovery. Always pair retries with exponential backoff, randomized jitter, and a maximum retry budget.

✓ PRODUCTION NOTE

Stateless services (no in-memory session/data that a specific request depends on) are what make horizontal scaling and rolling deploys simple — any instance can serve any request, and losing an instance costs nothing but that instance's in-flight requests. Session state, WebSocket connection state, and in-memory rate limiters are the usual things that accidentally make a service stateful; push them to Redis or a dedicated store instead of local memory.

+ TRICKY INTERVIEW QUESTION

Q: Design-style question: your API's main database is showing signs of overload (connection pool exhaustion, rising query latency) during traffic spikes. Walk through your options, roughly in the order you'd consider them.

A: A strong answer moves through layers rather than jumping straight to "add read replicas": (1) confirm it's actually a capacity problem and not a missing-index/inefficient-query problem — check slow query logs first, since scaling infrastructure to compensate for a bad query is expensive and doesn't actually fix it; (2) add caching in front of the hottest, most read-heavy queries to reduce DB load directly; (3) tune connection pool sizing relative to actual DB connection limits across all service instances (a common mistake: each of 10 service instances configured with a pool of 50 connections against a DB that only allows 100 total); (4) add read replicas for read-heavy workloads, routing reads away from the primary; (5) apply backpressure/load-shedding at the API layer (rate limiting, queueing, graceful degradation of non-critical features) so the database degrades predictably instead of falling over entirely under extreme spikes. The ordering signals whether a candidate reaches for infrastructure before diagnosis.

PART THREE

The Interview Bank

90+ medium-to-hard questions for a senior backend (JavaScript/Node.js) interview loop — conceptual, output-prediction, scenario-based, and a rapid-fire review round. Every answer explains the reasoning, not just the result.

- A Conceptual – JavaScript & Node Internals (24 questions)
- B Practical / Output-Based (22 questions)
- C Scenario-Based – System & Production Judgment (20 questions)
- D Rapid Fire – Lightning Round (26 questions)

A Conceptual — JavaScript & Node Internals

These test whether you understand *why* the language and runtime behave the way they do — the questions a senior candidate is expected to answer fluently, not just correctly.

1. Explain the difference between the call stack, the heap, and the event loop's task queues.

MEDIUM

CONCEPTUAL

The call stack tracks function execution frames — LIFO, synchronous. The heap is where objects are allocated (unordered memory pool). The task queues (microtask queue for Promises/nextTick, macrotask queues for timers/I/O/setImmediate) hold callbacks waiting for the stack to empty. The event loop's whole job is repeatedly checking "is the stack empty?" and, if so, pulling the next callback from the appropriate queue onto the stack.

2. What is the temporal dead zone, and why does it exist for `let`/`const` but not `var`?

MEDIUM

CONCEPTUAL

The TDZ is the span between entering a scope and the actual `let`/`const` declaration executing — accessing the variable in that span throws a `ReferenceError` rather than returning `undefined`. `var` declarations are hoisted and initialized to `undefined` immediately, so there's no window where access throws. The TDZ exists specifically to catch use-before-declaration bugs that `var`'s hoisting historically allowed to fail silently.

3. What's the difference between `Object.freeze`, `Object.seal`, and `Object.preventExtensions`?

MEDIUM

CONCEPTUAL

`preventExtensions` blocks adding new properties, but existing ones remain writable/configurable. `seal` additionally makes existing properties non-configurable (can't delete or redefine), but they stay writable. `freeze` additionally makes existing properties non-writable too — the strictest of the three. All three are shallow.

4. How does Node decide whether to use the microtask queue or a macrotask queue for a given async operation?

HARD

CONCEPTUAL

It's determined by the API, not a choice you make. Promise `.then`/`.catch`/`.finally` callbacks and `queueMicrotask` always go to the microtask queue. `process.nextTick` goes to its own even-higher-priority queue. `setTimeout`/`setInterval` go to the timers phase macrotask queue. `setImmediate` goes to the check phase. I/O callbacks go to the poll phase. The engine/runtime hard-codes this per API — understanding it is about memorizing the mapping, not reasoning about it per-call.

5. Why can't you `await` inside a regular (non-async) function's top level, and what were top-level `await` and IIFEs each designed to solve?

MEDIUM

CONCEPTUAL

`await` pauses execution of the enclosing `async function`, suspending its execution context to be resumed later — a plain synchronous function has no mechanism to suspend and resume like that. Async IIFEs (`(async () => { await ... })()`) were the historical workaround to run awaited code at a script's top level. Top-level `await` (ES2022, supported in ESM) removes the need for that wrapper, but only works in module context because it changes how the whole module's evaluation is scheduled relative to its importers.

6. What's the difference between a shallow copy and a deep copy, and name three ways to do each in modern JS.

MEDIUM

CONCEPTUAL

Shallow copy duplicates top-level properties only — nested objects are still shared references. Shallow: `spread ({...obj})`, `Object.assign({}, obj)`, `Array.prototype.slice()`. Deep: `structuredClone(obj)` (native, handles most types including circular refs), `JSON.parse(JSON.stringify(obj))` (loses functions, `undefined`, `Date` becomes a string, breaks on circular refs), or a recursive manual clone / library like `lodash's cloneDeep`.

7. Explain prototypal inheritance without using the word "class."

MEDIUM

CONCEPTUAL

Every object has an internal link (`[[Prototype]]`), accessible via `Object.getPrototypeOf` to another object. Property lookup walks this chain: if a property isn't found on the object itself, JS checks its prototype, then that prototype's prototype, and so on until `null`. `class` syntax is sugar over this same mechanism — it doesn't introduce a fundamentally different inheritance model, just a cleaner syntax for setting up the prototype chain.

8. What does Node's `require.cache` do, and what's one production-relevant consequence of it?

HARD

CONCEPTUAL

Node caches every module by resolved file path after the first `require()` — subsequent requires of the same path return the same cached exports object instead of re-executing the file. Consequence: modules are effectively singletons within a process, which is why a module-level `let count = 0` persists and is shared across every file that requires it — a common source of unexpected shared state, and also the basis for how singleton services (like a shared DB connection) are typically implemented in CJS.

9. What's the difference between `Array.prototype.map` and `Array.prototype.forEach` in terms of what you should use them for, including with async callbacks?

MEDIUM

CONCEPTUAL

`map` returns a new array of transformed values and should be used when you want that array. `forEach` returns `undefined` and is for side effects only. Neither `map` nor `forEach` awaits async callbacks — passing an `async` function to either still runs all callbacks concurrently/immediately (their returned promises are simply discarded/ignored), which is a common bug source when developers expect sequential `await`-per-item behavior. For that, use a `for...of` loop with `await`, or `Promise.all(arr.map(async ...))` for concurrent execution with a final awaited result.

10. What is a memory-safe way to implement a simple LRU cache, and why does insertion order matter for the data structure you'd choose?

HARD

CONCEPTUAL

JS `Map` preserves insertion order and offers $O(1)$ get/delete/set, making it a natural fit: on access, delete and re-insert the key to move it to the "most recently used" end; when size exceeds capacity, delete the first key (`map.keys().next().value`), which is the least recently used. A plain object doesn't guarantee the same ordering semantics for all key types (integer-like keys get reordered), which is why `Map` is preferred over an object for this specifically.

11. Why does Node recommend against synchronous file system calls (`fs.readFileSync`) in request-handling code paths?

MEDIUM

CONCEPTUAL

Synchronous fs calls block the entire event loop for their duration — no other request, timer, or I/O callback can run until it completes, even though disk I/O can take milliseconds under load or contention. It's acceptable at startup (reading config once, before serving traffic) but not inside a handler serving concurrent requests, where it directly degrades every other in-flight request's latency.

12. What's the difference between `null`, `undefined`, and an unset property, and how does `in` vs `hasOwnProperty` vs optional chaining relate?

MEDIUM

CONCEPTUAL

`undefined` means a variable/property has no assigned value (or doesn't exist at all); `null` is an explicit, intentional "no value" assignment. ``key' in obj`` checks the prototype chain too; `obj.hasOwnProperty('key')` checks only the object's own properties. `obj?.key`` safely reads a possibly-missing/nullish property without throwing, but doesn't distinguish "missing" from "present but `undefined`" — for that distinction you still need `in` or `hasOwnProperty`.

13. Explain debouncing vs throttling and give a backend (not just UI) use case for each.

MEDIUM

CONCEPTUAL

Debouncing delays execution until a quiet period has passed since the last call (resets the timer on every call) — a backend example is batching rapid successive writes to the same cache key into a single write after activity settles. Throttling guarantees execution happens at most once per fixed interval regardless of call frequency — a backend example is limiting outbound calls to a rate-limited third-party API to one per N milliseconds regardless of how fast your internal queue produces work.

14. What is currying, and is it something you'd actually use in a production Node codebase?

MEDIUM

CONCEPTUAL

Currying transforms a function taking multiple arguments into a sequence of functions each taking one argument, e.g. `add(a)(b)(c)`. In practice, it shows up in production code mostly for creating pre-configured/partially-applied functions — e.g. `const logError = createLogger('error')` — and in middleware factories (`rateLimit({ max: 100 })` returning the actual middleware function). It's a real, useful pattern in moderation, not something to reach for by default.

15. What does `structuredClone` support that `JSON.parse(JSON.stringify())` doesn't?

MEDIUM

CONCEPTUAL

`structuredClone` correctly handles circular references, `Map`, `Set`, `Date` (as an actual `Date`, not a string), `RegExp`, `ArrayBuffer`/typed arrays, and `Error` objects. It still cannot clone functions, DOM nodes (irrelevant on the backend), or class instances with private fields/prototypes (it clones the data, not the prototype). For plain-data backend objects it's almost always the better default over the JSON round-trip hack.

16. What's a race condition in a single-threaded JS environment, if there's no real concurrent thread execution?

HARD

CONCEPTUAL

Even without true parallel threads, interleaving between async operations creates race-condition-shaped bugs: if handler A reads a value, awaits I/O, then writes based on the stale read — and handler B modified that value during A's await — A's write can overwrite B's change or act on outdated state. It's not a CPU-level race, but the observable bug (lost update, inconsistent state from interleaved timing) is the same class of problem, and the fixes (locks, atomic DB operations, optimistic concurrency with version checks) are conceptually similar.

17. Why does Node.js use non-blocking I/O by default instead of the thread-per-request model common in other backend runtimes?

MEDIUM

CONCEPTUAL

Thread-per-request models spend real memory (thread stacks, typically 1-8MB each) and context-switching overhead per concurrent connection, which caps concurrency well below what's often achievable with a single-threaded event loop juggling non-blocking I/O — Node can hold tens of thousands of idle-but-open connections (e.g. long-polling, WebSockets) cheaply, because an idle connection just costs a small amount of tracked state, not a full OS thread. The tradeoff is that Node handles I/O-bound concurrency very well but needs deliberate offloading (worker threads, cluster) for CPU-bound work.

18. What's the difference between `Promise.race` and `Promise.any`?

MEDIUM

CONCEPTUAL

`Promise.race` settles as soon as ANY input promise settles — whether it resolves or rejects. `Promise.any` resolves as soon as any input promise resolves, but only rejects (with an `AggregateError`) if ALL of them reject. Use `race` for timeout patterns (racing a real call against a timeout that rejects); use `any` when you want the first successful result among several redundant attempts (e.g. querying multiple mirrored endpoints, first success wins).

19. What is monkey-patching, and what's a legitimate production use case versus a dangerous one?

MEDIUM

CONCEPTUAL

Monkey-patching means modifying an existing object/prototype's behavior at runtime, often one you don't own. Legitimate: APM/tracing libraries patching `http` or a DB driver's methods to auto-instrument calls for observability, in a controlled, well-tested way. Dangerous: patching built-ins like `Array.prototype` in application code — it can silently interact badly with third-party libraries that assume native behavior, and creates debugging nightmares since the modified behavior isn't visible at the call site.

20. Explain the difference between authentication and authorization, and where each typically lives in an Express middleware chain.

MEDIUM

CONCEPTUAL

Authentication confirms identity ("who are you") — typically an early middleware verifying a JWT/session and attaching `req.user`. Authorization confirms permission ("are you allowed to do this specific thing") — typically applied per-route or per-resource, often after authentication, checking roles/ownership against the specific action being performed. Conflating them (e.g. treating "logged in" as sufficient for any action) is the root cause of broken object-level authorization bugs.

21. What does idempotency mean for an API endpoint, and why does it matter specifically for payment/write-heavy endpoints?

MEDIUM

CONCEPTUAL

An idempotent operation produces the same end result no matter how many times it's applied. For write endpoints exposed to unreliable networks (mobile clients, retried requests after a timeout), idempotency prevents a retried request from double-charging a card or double-creating a resource — typically implemented via a client-supplied idempotency key that the server checks before processing, returning the original result for a duplicate key instead of re-executing the operation.

22. What's the difference between vertical and horizontal partitioning/sharding, at a conceptual level a backend engineer should know even without deep DB specialization?

HARD

CONCEPTUAL

Vertical partitioning splits by columns/fields (e.g. moving rarely-accessed large text fields to a separate table/store to keep the hot table smaller). Horizontal partitioning (sharding) splits by rows across multiple databases/instances, typically by a shard key (e.g. user ID range or hash) — each shard holds a subset of rows for all columns. Sharding solves scale (data/traffic exceeding one machine) but introduces cross-shard query complexity; vertical partitioning solves hot-table bloat without that complexity.

23. What's the difference between optimistic and pessimistic locking, and when would you choose each in a Node service?

HARD

CONCEPTUAL

Pessimistic locking acquires a lock before reading/modifying a resource, blocking other writers until release — safe but reduces concurrency and risks deadlocks/held locks on a crashed process. Optimistic locking reads a version/timestamp, allows concurrent reads, and only rejects the write at commit time if the version changed since the read (a compare-and-swap). Optimistic is generally preferred for typical web API write patterns (low actual contention, want high read concurrency); pessimistic makes sense for high-contention resources where retries under optimistic locking would be frequent and costly.

24. What is a memory leak, precisely — define it in terms of reachability, not "using too much memory."

MEDIUM

CONCEPTUAL

A memory leak is memory that remains reachable from a GC root (and therefore never collected) but is no longer actually needed by the program's logic. It's not about total memory usage — a program legitimately holding a large in-memory cache isn't leaking. It's specifically about references that outlive their intended purpose: a forgotten listener, a growing collection nothing ever prunes, a closure retaining data it no longer needs — reachable, but dead weight.

B

Practical / Output-Based — "What does this print?"

These test precise mental execution of real JS semantics. Read each snippet carefully before checking the answer — the whole point is training your ability to trace execution, not recognize a pattern.

25. What does this log?

MEDIUM

OUTPUT

```
console.log(typeof null);
console.log(typeof undefined);
console.log(typeof function(){});
console.log(typeof []);
console.log(null instanceof Object);
```

```
'object' 'undefined' 'function' 'object' false
```

`typeof null` being `'object'` is a famous historical bug preserved for backward compatibility. Functions get their own `typeof` result despite being objects under the hood. Arrays are `typeof 'object'` too — use `Array.isArray()` to actually distinguish them. `null instanceof Object` is `false` because `instanceof` checks the prototype chain, and `null` has none.

26. What's the output order?

HARD

OUTPUT

```
async function a() { console.log('a1'); await b(); console.log('a2'); }
async function b() { console.log('b1'); }

console.log('script start');
setTimeout(() => console.log('timeout'), 0);
a();
Promise.resolve().then(() => console.log('promise'));
console.log('script end');
```

```
script start a1 b1 script end a2 promise timeout
```

`a()` runs synchronously until its first `await`, logging ``a1``, then calls `b()` which runs fully synchronously (no `await` inside it) logging ``b1``, then `a()` awaits `b()`'s returned promise — scheduling the rest of `a` as a microtask. Sync code continues: ``script end``. Microtask queue drains: ``a2`` (was scheduled first), then ``promise``. Finally the macrotask: ``timeout``.

27. What does this print?

MEDIUM

OUTPUT

```
const arr = [1, 2, 3];
arr.length = 0;
console.log(arr);
console.log(arr[0]);

const obj = { a: 1 };
const obj2 = obj;
obj2.a = 2;
console.log(obj.a);
```

```
[] undefined 2
```

Setting `.length = 0` on an array truncates it in place — a real, mutating operation, not a no-op. Objects are assigned by reference, so `obj2` and `obj` point to the same underlying object; mutating through either name affects both.

28. What does this print, and why?

HARD

OUTPUT

```
function Counter() {
  this.count = 0;
  this.increment = function() { this.count++; };
}
const c = new Counter();
const inc = c.increment;
try { inc(); } catch (e) { console.log(e instanceof TypeError); }
console.log(c.count);
```

```
true 0
```

Extracting `c.increment` as a bare reference and calling it loses its receiver — inside strict-mode class/module code, `this` is `undefined`, so `this.count++` throws a `TypeError` trying to read `count` off `undefined`. `c.count` remains `0` because `increment()` never actually completed against `c`.

29. What does this log?

MEDIUM

OUTPUT

```
console.log([1, 2, 3] + [4, 5, 6]);
console.log(1 + '1');
console.log('5' - 1);
console.log('5' + 1);
console.log([] + {});
console.log({} + []);
```

```
'1,2,34,5,6' '11' 4 '51' '[object Object]' '[object Object]'
```

+ on arrays/objects triggers coercion to primitives via `toString()` first (arrays join with commas, plain objects become `'[object Object]'`), then string concatenation if either side is a string. `-` always coerces both sides to numbers, since there's no meaningful "subtraction" for strings.

30. Trace this generator — what does each `.next()` call return?

HARD

OUTPUT

```
function* gen() {
  const x = yield 1;
  const y = yield x + 1;
  return x + y;
}
const g = gen();
console.log(g.next());      // ?
console.log(g.next(10));    // ?
console.log(g.next(20));    // ?
```

```
{ value: 1, done: false } { value: 11, done: false } { value: 30, done: true }
```

The first `.next()` runs to the first `yield 1` and pauses, returning `1`. The value passed into the *second* `.next(10)` becomes the result of the `yield` expression that was paused (`x = 10`), then execution continues to `yield x + 1` = `11`. The third `.next(20)` resumes with `y = 20`, executing `return x + y` = `30`, and `done` flips to `true`.

31. What's logged, and in what order?

HARD

OUTPUT

```
for (let i = 0; i < 3; i++) {  
  Promise.resolve().then(() => console.log('promise', i));  
  setTimeout(() => console.log('timeout', i), 0);  
}  
console.log('sync');
```

```
sync promise 0 promise 1 promise 2 timeout 0 timeout 1 timeout 2
```

All three loop iterations run synchronously first, scheduling 3 microtasks and 3 macrotasks (each with its own `let i` binding, correctly captured). Sync code finishes (`'sync'`). Then ALL queued microtasks drain before the event loop touches a single macrotask — so all three `'promise'` logs print before any `'timeout'` log, even though they were interleaved in scheduling order in the loop.

32. What does this output?

MEDIUM

OUTPUT

```
const a = { valueOf: () => 5 };  
const b = { toString: () => '10' };  
console.log(a + 1);  
console.log(b + 1);  
console.log(a == 5);
```

```
6 '101' true
```

Coercion prefers `valueOf()` over `toString()` when converting an object to a primitive for numeric-flavored operations. `a` has `valueOf` returning `5`, so `a + 1` numerically adds to `6`. `b` only has `toString`, so it coerces to the string `'10'`, and `+` with a string operand does concatenation: `'101'`. `a == 5` triggers the same `valueOf`-based coercion for the loose equality check.

33. What happens when this runs?

HARD

OUTPUT

```
async function getData() {  
  throw new Error('fail');  
}  
getData();  
console.log('after call');
```

after call (then, shortly after, in a separate crash/log) UnhandledPromiseRejection: Error: fail

`getData()` returns a rejected promise (since throwing inside an `async` function rejects its returned promise rather than throwing synchronously) — but nothing awaits or `.catch`'es it. `'after call'` logs immediately, synchronously. The unhandled rejection is reported afterward, and in modern Node (default since v15) it terminates the process with a non-zero exit code unless a handler is registered.

34. What's the result?

MEDIUM

OUTPUT

```
const obj = { a: 1, b: 2, a: 3 };  
console.log(obj);  
  
const arr = [1, , 3]; // note the hole  
console.log(arr.length);  
console.log(arr.map(x => x * 2));
```

{ a: 3, b: 2 } 3 [2, <1 empty item>, 6]

Duplicate keys in an object literal: the last one wins silently, no error. Array "holes" (sparse arrays) are distinct from actual `undefined` elements — `.map()` (and `.forEach`, `.filter`) skip holes entirely rather than invoking the callback with `undefined`, which is a genuinely easy-to-miss distinction from a normal array containing explicit `undefined` values.

35. Trace the output of this class hierarchy.

HARD

OUTPUT

```
class Base {
  constructor() { console.log('Base'); this.init(); }
  init() { console.log('Base.init'); }
}
class Derived extends Base {
  init() { console.log('Derived.init'); }
}
new Derived();
```

Base Derived.init

Method resolution happens through the prototype chain at call time, not at definition time — even though `Base`'s constructor calls `this.init()`, `this` is a `Derived` instance, so `Derived.init` (the overridden method) runs, not `Base.init`. This is the same dynamic-dispatch behavior as regular prototype method overriding, and it's a classic gotcha: calling an overridable method from a base constructor before a subclass has finished its own setup can run against half-initialized subclass state.

36. What does this log — walk through the destructuring carefully.

MEDIUM

OUTPUT

```
const { a, b = 10, ...rest } = { a: 1, c: 3, d: 4 };
console.log(a, b, rest);

const [x, , z] = [1, 2, 3];
console.log(x, z);
```

1 10 { c: 3, d: 4 } 1 3

b isn't present in the source object, so its default value `10` applies. `...rest` collects every remaining enumerable own property not already destructured. Array destructuring can skip positions with an empty slot in the pattern (`[x, , z]`), simply ignoring index 1.

37. What's printed, in order, and why does it matter for error-handling code?

HARD

OUTPUT

```
function test() {  
  try {  
    return 'try';  
  } finally {  
    console.log('finally ran');  
  }  
}  
console.log(test());
```

```
finally ran try
```

`finally` always runs before the function actually returns, even though a `return` statement already executed in the `try` block — the return value is computed and "queued," but `finally` gets a chance to run (and, notably, could even override the return value with its own `return`, which is a real footgun worth knowing about but avoiding in practice).

38. What does this print?

MEDIUM

OUTPUT

```
const map = new Map();  
map.set('1', 'string key');  
map.set(1, 'number key');  
console.log(map.size);  
console.log(map.get('1'), map.get(1));  
  
const obj = {};  
obj[1] = 'a';  
obj['1'] = 'b';  
console.log(Object.keys(obj).length);
```

```
2 'string key' 'number key' 1
```

`Map` keys preserve their original type — `'1'` (string) and `1` (number) are distinct keys, unlike plain object keys, which are always coerced to strings. So `obj[1]` and `obj['1']` collide into the same key and the second write overwrites the first.

39. What's the output, and what Node-specific behavior does it demonstrate?

HARD

OUTPUT

```
// emitter.js
const { EventEmitter } = require('events');
const emitter = new EventEmitter();

emitter.on('data', () => console.log('listener 1'));
emitter.emit('data');
emitter.on('data', () => console.log('listener 2'));
emitter.emit('data');
```

```
listener 1 listener 1 listener 2
```

`emit()` is synchronous and calls whatever listeners are registered *at that exact moment* — it does not wait for or discover listeners added afterward. The first `emit` only triggers listener 1 (listener 2 wasn't registered yet); the second `emit` triggers both, in registration order.

40. What does this print, and what production pattern does it illustrate?

HARD

OUTPUT

```
function delay(ms, value) {
  return new Promise(resolve => setTimeout(() => resolve(value), ms));
}

async function run() {
  const results = [];
  for (const ms of [300, 100, 200]) {
    results.push(await delay(ms, ms));
  }
  console.log(results, 'sequential');
}

async function runParallel() {
  const results = await Promise.all([300, 100, 200].map(ms => delay(ms, ms)));
  console.log(results, 'parallel');
}

run(); // finishes after ~600ms total (300+100+200, sequential)
runParallel(); // finishes after ~300ms total (max of the three, concurrent)
```

```
[300, 100, 200] 'parallel' (logs first, ~300ms in) [300, 100, 200] 'sequential' (logs second, ~600ms in)
```

Both arrays print in original order regardless of individual delay — `await` in the loop preserves iteration order for pushing results, and `Promise.all` preserves input-array order regardless of resolution order. The real lesson is timing: sequential `await` in a loop pays the sum of all delays; `Promise.all` pays only the max, since the promises are all created (and their timers start) before any awaiting happens.

41. What's wrong with this rate-limiter and what would actually happen under concurrent requests?

HARD

OUTPUT

```
let requestCount = 0;
const LIMIT = 100;

app.use(async (req, res, next) => {
  const current = await redis.get('count');
  if (Number(current) >= LIMIT) return res.status(429).end();
  await redis.set('count', Number(current) + 1);
  next();
});
```

Under concurrent requests, the count can exceed LIMIT — multiple requests read the same "current" value before any of them writes the incremented value back.

This is a classic read-then-write race: between the `redis.get` and `redis.set` (both async, both involve network round trips with real latency), many concurrent requests can all read the same stale count and all pass the check before any of them writes an updated value — the check-then-act sequence isn't atomic. The fix is using Redis's atomic `INCR` (a single atomic operation, no separate read-then-write) combined with checking the returned value, or a Lua script/`MULTI` transaction for anything more complex than a simple counter.

42. What does this print, and why is it a footgun in real code?

MEDIUM

OUTPUT

```
console.log(0.1 + 0.2);
console.log(0.1 + 0.2 === 0.3);
console.log((0.1 + 0.2).toFixed(2));
console.log(9007199254740993 === 9007199254740992 + 1);
```

0.30000000000000004 false '0.30' false

IEEE-754 double-precision floats can't represent 0.1 or 0.2 exactly, so their sum has a tiny representational error. `.toFixed()` rounds for display, masking (not fixing) the underlying imprecision. The last line demonstrates `Number.MAX_SAFE_INTEGER` ($2^{53} - 1$) territory — beyond that, integers can't be represented exactly either, which is why financial/large-ID systems often use `BigInt` or string representations for values that exceed it.

43. What happens when you run this, and what does it demonstrate about `try/catch` and async iteration?

HARD

OUTPUT

```
async function processAll(items) {
  const results = [];
  for (const item of items) {
    try {
      results.push(await risky(item));
    } catch (e) {
      results.push({ error: e.message });
    }
  }
  return results;
}
// vs.
async function processAllBroken(items) {
  try {
    return await Promise.all(items.map(risky));
  } catch (e) {
    return { error: e.message }; // loses ALL successful results
  }
}
```

`processAll`: returns partial results, one error object per failed item. `processAllBroken`: on ANY single failure, discards every successful result too – `Promise.all` rejects as a whole on the first rejection.

Wrapping each `await` individually in a loop preserves per-item outcomes; wrapping a `Promise.all` in one `try/catch` is all-or-nothing — a single failure anywhere in the batch loses every successful result that would otherwise have been available. This is a very common real bug in batch-processing endpoints, and it's exactly the failure mode `Promise.allSettled` (Topic 04) is designed to avoid.

44. What's the output, and what does it reveal about how Node handles `process.exit()`?

HARD

OUTPUT

```
console.log('before');
fs.writeFile('/tmp/out.txt', 'data', () => {
  console.log('file written');
});
process.exit(0);
console.log('after');
```

before (process exits immediately — 'file written' and 'after' never print; the file write may or may not have completed)

`process.exit()` terminates immediately and synchronously — it does not wait for pending I/O callbacks (the `fs.writeFile` callback), pending timers, or even the rest of the current synchronous script (`console.log('after')` never runs). This is exactly why graceful shutdown (Topic 14) explicitly drains and awaits cleanup work before calling `process.exit()`, rather than calling it immediately on receiving a shutdown signal.

45. What's the output? (`in` operator and prototype pollution shape)

HARD

OUTPUT

```
const malicious = JSON.parse('{"__proto__": {"isAdmin": true}}');
const merged = Object.assign({}, malicious);
const freshObj = {};
console.log(freshObj.isAdmin);
```

undefined (with `JSON.parse` + `Object.assign`, as shown here — safe)

This specific combination is actually safe: `JSON.parse` creates `__proto__` as an own, ordinary property (not the special prototype-linking accessor), and `Object.assign` copies own enumerable properties without triggering prototype mutation. The *real* vulnerability shows up with naive recursive-merge utilities that do `target[key] = source[key]` for every key including `__proto__` without guarding against it — that pattern DOES let `__proto__` writes pollute `Object.prototype` globally. The interview point: know precisely which merge patterns are vulnerable rather than assuming all JSON-derived merging is dangerous, or safe.

46. What does this log, and what's the lesson for writing middleware?

MEDIUM

OUTPUT

```
app.use((req, res, next) => {
  console.log('middleware 1 start');
  next();
  console.log('middleware 1 end');
});
app.use((req, res) => {
  console.log('middleware 2');
  res.send('done');
});
```

```
middleware 1 start middleware 2 middleware 1 end
```

`next()` is synchronous and hands control to the next middleware immediately, but execution returns to the code *after* `next()` in the calling middleware once the downstream chain completes synchronously — this is standard callback-stack behavior, not async. It matters in practice for logging/timing middleware wrapped around `next()`: code after `next()` runs after the whole downstream chain, which is exactly how response-time-logging middleware is built.

C Scenario-Based — System & Production Judgment

These mirror how senior interviews actually probe judgment: a realistic situation, often deliberately underspecified, where the interviewer wants to see your diagnostic process as much as your final answer.

47. Your Node API's memory usage grows by roughly 50MB every hour and never comes back down, even during low-traffic periods. Walk through your investigation.

HARD

SCENARIO

First, confirm it's actually a leak and not healthy sawtooth GC behavior (Topic 15) — check the trend over a longer window, not a single hour. If it's real: check `process.memoryUsage()` breakdown (heap vs external vs rss) to know whether it's JS-heap or native/Buffer growth. For JS-heap growth, take two heap snapshots hours apart and diff retained size by constructor — the object type with runaway count is almost always the lead. Cross-reference against the four usual suspects: unbounded caches, forgotten timers, accumulating event listeners, and closures over large data. Reproduce locally under synthetic sustained load if possible, since some leaks only manifest at a request rate that triggers a particular code path repeatedly.

48. A junior engineer wants to cache API responses in a module-level plain object to "speed things up." What questions do you ask before approving this in review?

MEDIUM

SCENARIO

Does the cache have a bounded size or eviction policy, or will it grow unboundedly under real traffic (a leak by construction)? Does it need to be consistent across multiple instances (if so, a per-process in-memory cache will serve stale/inconsistent data depending on which instance handles a request — Topic 13)? What's the invalidation strategy when underlying data changes — is a short TTL acceptable, or does correctness require active invalidation? Is the cached data ever user-specific or sensitive, which would make a shared module-level cache a data-leak risk across requests/users?

49. Your service's p99 latency doubled after the last deploy, but average latency looks unchanged, and no errors increased. What do you check first?

HARD

SCENARIO

Diff the deploy for anything that could add a rarely-hit slow path: a new code branch handling an edge case with an unindexed query, a new dependency call added to a specific route, or a new synchronous operation (JSON parsing a larger payload, a new regex) that only bites requests with larger-than-typical input. Segment the latency regression by route and by request-payload characteristics rather than looking at the aggregate — since average is unchanged, the regression is concentrated in a specific, less-common code path or input shape, not distributed evenly.

50. You're designing a webhook receiver endpoint for a third-party payment provider. What production concerns does this specific kind of endpoint raise that a typical internal API doesn't?

HARD

SCENARIO

Signature verification (confirm the payload actually came from the provider, using their HMAC signature scheme, before trusting anything in it) — done before any business logic runs. Idempotency (providers commonly retry webhook delivery on any non-2xx or timeout, so the same event may arrive multiple times — process by event ID with a dedup check, not blindly). Fast acknowledgment (respond 200 quickly and do heavy processing asynchronously via a queue, since providers often have tight response-time expectations and will treat a slow response as a failure needing retry). Raw body access (signature verification typically needs the raw, unparsed request body — a body-parsing middleware that runs first can break this if it doesn't preserve the raw buffer).

51. A background job that processes a queue occasionally processes the same message twice. The team wants a quick fix. What's your actual recommendation?

MEDIUM

SCENARIO

Most queueing systems provide at-least-once delivery by design (message redelivered if not acknowledged in time, or on consumer crash before ack) — duplicate delivery is expected behavior, not a bug to eliminate at the queue level. The correct fix is making the job handler idempotent: track processed message IDs (with a TTL matching the queue's redelivery window) and short-circuit on a duplicate, or design the operation itself to be naturally idempotent (e.g. an upsert keyed by message ID rather than an insert). Trying to force exactly-once delivery at the queue layer is usually the wrong direction — that's a much harder and more expensive guarantee to build than idempotent consumers.

52. You need to add a new required field to an API response that thousands of existing mobile clients consume, some of which are years out of date and won't be updated. How do you roll this out?

MEDIUM

SCENARIO

Add the field as purely additive (new optional field in the response), never remove or rename existing fields, and never make old clients' assumptions break — old clients that don't know about the new field will simply ignore it if using typical JSON parsing. For any genuinely breaking change (removing a field, changing a type), version the API explicitly (URL path or header-based versioning) and maintain the old version until client adoption data shows it's safe to sunset — "we'll just change it" is not viable when you don't control the client release cycle.

53. Your team is debating REST vs GraphQL for a new internal API with roughly 15 downstream frontend consumers, each needing different subsets of a complex data model. What factors actually matter for this decision?

HARD

SCENARIO

GraphQL's core advantage is exactly this shape of problem — many consumers each needing different field subsets of overlapping underlying data, avoiding both over-fetching (REST returning fixed shapes with unused fields) and the alternative of maintaining dozens of bespoke REST endpoints per consumer need. The real costs to weigh: N+1 query risk is more insidious in GraphQL resolvers (needs DataLoader-style batching discipline from day one), caching is harder (no free HTTP-level caching by URL the way REST gets), and the team's existing operational familiarity/tooling maturity matters — a team with zero GraphQL production experience taking this on for a first-of-its-kind internal service has real ramp-up cost that a pure technical comparison ignores.

54. A downstream service your API depends on has become slow (not down, just slow — 8 second p50 instead of its usual 200ms). Your API doesn't have a timeout configured on that call. What happens, concretely, and how do you prevent it going forward?

HARD

SCENARIO

Without a timeout, your requests to the slow downstream hold open connections/pending promises for the full 8 seconds each — under any real concurrent traffic, this exhausts your service's own capacity (connection pool to the downstream, and potentially the request-handling capacity itself if requests queue behind it), turning one slow dependency into a full outage of your service, not just degraded latency. Fix: always set explicit timeouts on outbound calls (shorter than your own service's SLA), pair with a circuit breaker (Topic 20) so repeated timeouts stop attempting the call entirely for a cooldown period, and have a defined fallback behavior (cached/stale data, a degraded response, or a clear error) for when the circuit is open.

55. You're asked to design the token-refresh flow for a Node API using short-lived JWT access tokens and long-lived refresh tokens. What are the key production decisions?

HARD

SCENARIO

Where refresh tokens are stored (httpOnly, secure cookies are generally safer than client-accessible storage, since JS-readable storage is vulnerable to XSS-based token theft). Whether refresh tokens are rotated on each use (issuing a new refresh token per refresh, invalidating the old one — this lets you detect token theft: if a stolen, already-used refresh token is replayed, you can revoke the whole token family). How revocation works at all — pure stateless JWTs can't be individually revoked before expiry, so a real system usually needs a server-side revocation list or short-enough access-token TTL that revocation delay is acceptable. Handling concurrent refresh requests from the same client (e.g. multiple tabs) without each one invalidating the other's newly-rotated token in a race.

56. Your team's Node service occasionally restarts unexpectedly in production with no application-level error logged. Where do you look?

HARD

SCENARIO

Check the orchestrator/container events first, not application logs — a container OOM-kill (exceeding its memory limit) produces zero application-level error output, since the process is killed externally, not by its own error handling. Check `kubectl describe pod` (or equivalent) for OOMKilled status and exit codes. Also check for liveness probe failures causing orchestrator-initiated restarts (the app was technically fine, but a health check misconfiguration or a slow endpoint used as the health check caused a false-positive restart). Only after ruling both out would you suspect a native-level crash (a segfault in a native addon) or an unhandled signal.

57. You're building a CSV export endpoint that can generate files from 100 rows up to 2 million rows depending on the customer. What's your approach, and why does it change based on scale?

MEDIUM

SCENARIO

Building the full CSV string in memory and returning it works fine at 100 rows but risks OOM and a very long time-to-first-byte at 2 million rows. The scalable approach streams: query the DB with a cursor/streaming read (not `.find().toArray()` loading everything at once), transform rows to CSV incrementally through a Transform stream, and pipe directly to the HTTP response — memory stays bounded regardless of row count, and the client starts receiving data immediately instead of waiting for full generation. For very large exports, an async job that generates the file to object storage and emails/notifies a download link is often the right answer, since holding an HTTP connection open for a multi-minute export is fragile against client timeouts and network interruptions.

58. A code reviewer flags that a new endpoint doesn't validate the `Content-Type` header before parsing the body. Why does this matter?

MEDIUM

SCENARIO

Body-parsing middleware (like `express.json()`) typically only activates for matching `Content-Type` headers by design — so this is usually a non-issue for standard middleware. But for custom parsing logic, skipping the check means accepting and attempting to parse arbitrary content as JSON regardless of what the client actually sent, which can produce confusing errors, and in some setups opens a route for content-type confusion attacks (e.g. bypassing a WAF or proxy rule that only inspects requests with a specific declared content type). The safe default is to explicitly validate expected content type and reject mismatches with a clear 415 status rather than attempting to parse anyway.

59. Your team wants to migrate a large MongoDB collection's schema (adding a required field with a computed default) with zero downtime, while the service keeps reading/writing normally. What's your rollout plan?

HARD

SCENARIO

Multi-phase: (1) deploy application code that can handle BOTH the presence and absence of the new field (read with a fallback default, write the new field going forward) — this must ship and be fully live before touching existing data; (2) run a backfill migration against existing documents, in batches, with rate limiting to avoid overwhelming the DB, and idempotent so it's safely re-runnable if interrupted; (3) once backfill is confirmed complete, optionally deploy a follow-up version that assumes the field is always present, removing the fallback logic. Skipping step 1 (deploying code that requires the field before data has it) is the most common way this kind of migration causes an outage.

60. A load test shows your Node service's throughput plateaus at a certain concurrency level and doesn't improve with more concurrent client connections. CPU usage on the box is only at 40%. What's your hypothesis?

HARD

SCENARIO

Since CPU isn't saturated, the bottleneck is elsewhere — most likely a fixed-size resource being exhausted: the DB connection pool (requests queueing for a free connection once the pool is full, regardless of how much more the CPU could handle), the libuv thread pool if the workload touches file I/O or certain crypto calls (Topic 11), or an external dependency's own rate limit/concurrency cap. The diagnostic move is checking pool utilization and queue-wait metrics specifically, not just CPU/memory — a single-threaded event loop with low CPU usage under high concurrency is a strong signal that something else is the actual ceiling.

61. You inherit a Node service where all configuration (DB credentials, API keys, feature flags) lives in a single large JSON file committed to the repo. What do you change, and in what order?

MEDIUM

SCENARIO

Immediate priority: rotate any real secrets that were ever committed (assume they're compromised — git history retains them even after later removal) and move secrets to a proper secrets manager or environment variables injected at deploy time, never committed. Separately, non-secret configuration (feature flags, non-sensitive settings) can reasonably stay in versioned config files, but should be separated from secrets structurally so the two don't get re-mixed by accident later. This is a security-first prioritization question — reviewers are checking that you rotate compromised secrets before doing anything else, not just "fix the architecture."

62. Your API returns different response shapes for the same endpoint depending on an internal feature flag, and a frontend team is intermittently seeing runtime errors they can't reproduce consistently. What's your first move?

MEDIUM

SCENARIO

Ask which feature flag(s) affect this endpoint's response shape and whether the flag can vary per-request (per-user rollout percentage, per-tenant) — that's almost certainly the source of the "intermittent, unreproducible" pattern, since the frontend is likely only handling one of the shapes. The fix is either making the response shape flag-independent (additive-only changes regardless of flag state) or having the frontend explicitly branch on a shape-indicating field in the response rather than assuming one fixed shape — the deeper lesson is that server-side feature flags affecting response contracts need to be treated as API versioning, not an invisible internal detail.

63. Describe how you'd design health check endpoints (`/healthz``, `/readyz``) for a Node service with a database and a message queue dependency.

MEDIUM

SCENARIO

Liveness (`/healthz``) should check only "is this process fundamentally alive and not deadlocked" — a trivial check with no external dependency calls, since a liveness probe failing triggers a restart, and you don't want a transient DB blip to cause a restart storm that doesn't actually fix anything. Readiness (`/readyz``) should check that critical dependencies (DB connection, queue connection) are actually reachable, since a readiness failure just stops new traffic routing without restarting the process — appropriate for "temporarily can't serve requests well" versus liveness's "this process itself is broken."

64. Your team ships a hotfix directly to production without going through staging, because "it's a one-line change." It breaks a different, unrelated endpoint. What process gap does this reveal, and what's the fix that doesn't just slow every deploy down?

MEDIUM

SCENARIO

The gap isn't really "no staging step" — it's that the change bypassed automated regression coverage that would have caught the unrelated breakage regardless of environment. The proportionate fix is ensuring CI (unit/integration tests, even a fast subset) runs on every change including hotfixes, rather than adding a mandatory manual staging soak for every deploy, which would slow down all future work including genuinely safe changes. A useful middle ground many teams use: hotfixes still go through CI and a fast smoke-test suite, but skip a longer manual QA cycle — fast automated safety nets, not necessarily full process parity with a normal release.

65. A teammate proposes storing session data in a JWT itself (instead of a server-side session store) to "simplify" the architecture and avoid a Redis dependency. What trade-offs do you raise?

HARD

SCENARIO

JWTs genuinely remove the Redis dependency and the associated operational overhead, and they scale trivially since any instance can verify a token without a shared store. The real costs: revocation becomes hard (you can't invalidate a specific JWT before its expiry without reintroducing some server-side state — a denylist — which partially defeats the purpose); token size grows with session data, adding overhead to every single request; and any session data embedded in the token is only as fresh as when it was issued, so "logged out everywhere" or "permissions changed" scenarios need careful handling. This is a legitimate trade-off, not a clear-cut mistake — the right call depends on whether the product actually needs fast, precise revocation (e.g. security-sensitive apps usually do).

66. You're asked to add distributed tracing to a system of 6 Node microservices that currently only have per-service logs. What's the core mechanism that makes a trace actually "distributed," and what's the minimum you need each service to do?

HARD

SCENARIO

The core mechanism is context propagation: a trace ID (and typically a parent span ID) generated at the edge of the system must be passed forward through every downstream call — as a header on outbound HTTP calls, as metadata on queue messages — so every service handling any part of that original request can tag its own logs/spans with the same trace ID. The minimum per-service requirement: read an incoming trace context header if present (or generate one if this service is the entry point), include it in all outbound calls it makes, and include it in its own structured logs. Without consistent propagation through every hop — including message queues, which are easy to forget — the trace breaks at that hop and you lose the ability to correlate across it.

D Rapid Fire — Lightning Round

Fast, precise questions for last-minute review — the kind an interviewer drops in quickly to check breadth. Aim to answer each in one or two sentences before reading the answer.

67. Why is `Array.isArray(arr)` preferred over `typeof arr === 'object'`?

MEDIUM

RAPID

`typeof` returns `'object'` for arrays too (they're objects under the hood) — it can't distinguish an array from a plain object or `null`-derived object. `Array.isArray` checks the actual internal class specifically.

68. What does `Object.entries()` return, and what's it commonly used for in Node code?

MEDIUM

RAPID

An array of `[key, value]` pairs for an object's own enumerable properties — commonly used to iterate an object with `for...of` or transform it via `.map`/`.filter`/`.reduce`, which plain objects don't support directly.

69. What's the difference between `slice` and `splice` on arrays?

MEDIUM

RAPID

`slice` is non-mutating and returns a new array subset. `splice` mutates the original array in place (removing/inserting elements) and returns the removed elements.

70. What is a "dirty read" and how does it relate to database transaction isolation levels?

HARD

RAPID

Reading data written by another transaction that hasn't committed yet — possible under the weakest isolation level (Read Uncommitted). If that other transaction rolls back, you've read data that never truly existed. Most production systems default to at least Read Committed to prevent this.

71. What does `NODE_ENV=production` actually change, concretely, in a typical Express app?

MEDIUM

RAPID

Express itself disables view-caching bypass and verbose error pages, and many libraries (e.g. some ORMs, React in an SSR context) branch on it for performance optimizations and stripping debug output. It's a convention, not a language feature — it does nothing automatically unless your code or dependencies explicitly check it.

72. What's the difference between `PATCH` and `PUT` in REST semantics?

MEDIUM

RAPID

`PUT` is a full replacement of the resource (idempotent, sends the entire representation). `PATCH` is a partial update (sends only the changed fields) — also expected to be idempotent in practice, though the spec allows non-idempotent `PATCH` semantics.

73. Why does `for...in` iterate array indices as strings, and why is `for...of` generally preferred for arrays?

MEDIUM

RAPID

`for...in` iterates enumerable property keys (including inherited ones, and array indices are just string keys under the hood) — it's designed for objects, not sequences. `for...of` iterates actual values via the iterator protocol and is the correct tool for arrays.

74. What's a "poison pill" message in a queue-processing context?

HARD

RAPID

A message that consistently crashes or fails the consumer no matter how many times it's retried (e.g. malformed data triggering an unhandled exception) — without a dead-letter queue and a retry limit, it can block or endlessly cycle through the queue, sometimes taking down the whole consumer repeatedly.

75. What's the difference between `==` null checks and `??` (nullish coalescing)?

MEDIUM

RAPID

`value ?? fallback` uses `fallback` only when `value` is `null` or `undefined` — unlike `||`, it does NOT fall through for other falsy values like `0`, `""`, or `false`, which is usually what you actually want for defaults.

76. What is a "sticky session," and why does it complicate horizontal scaling?

MEDIUM

RAPID

Routing all requests from a given client to the same backend instance (often for in-memory session/WebSocket state). It complicates scaling because it ties load distribution to client identity rather than current load, and losing that instance loses that client's state — the better fix is usually externalizing the state instead of relying on stickiness.

77. What does `Array.prototype.reduce`'s second argument (initial value) protect against if omitted?

MEDIUM

RAPID

Without an initial value, `reduce` uses the array's first element as the initial accumulator and starts iterating from the second — this throws on an empty array (`TypeError: Reduce of empty array with no initial value`) and can produce a wrongly-typed accumulator on a non-empty array. Always pass an explicit initial value.

78. What's the difference between a 502 and a 504 error?

MEDIUM

RAPID

502 Bad Gateway means an upstream/proxy got an invalid response from the backend (e.g. the backend crashed or reset the connection). 504 Gateway Timeout means the upstream didn't get a response from the backend in time at all — the backend may still be processing.

79. What does "eventual consistency" mean, and name one Node/backend scenario where you'd deliberately accept it.

MEDIUM

RAPID

A system guarantees that, absent new writes, all reads will *eventually* reflect the latest write — but not necessarily immediately. Example: a search index updated asynchronously after a DB write — accepting a brief window where search results lag behind the source of truth, in exchange for not blocking the write path on reindexing.

80. What is `Buffer` in Node, and why isn't it just a regular array of numbers?

MEDIUM

RAPID

`Buffer` represents raw binary data, allocated outside the V8 JS heap for efficiency with file/network I/O. It behaves array-like for byte access but is backed by fixed-size raw memory, not a resizable JS array — this is also why Buffer memory shows up under `external`, not `heapUsed`, in `process.memoryUsage()`.

81. What's the purpose of a dead letter queue (DLQ)?

MEDIUM

RAPID

A separate queue where messages that repeatedly fail processing (exceeding a retry limit) get routed instead of endlessly retrying or being silently dropped — preserves the failed message for manual inspection/reprocessing without blocking the main queue.

82. Why is `JSON.stringify` on an object with a circular reference a runtime error, and what's the quick fix?

MEDIUM

RAPID

It throws `TypeError: Converting circular structure to JSON` because naive recursive serialization would loop forever. Fixes: a custom replacer function that tracks visited objects, a library like `flatted`, or (in modern environments) restructuring the data to avoid the cycle in the first place.

83. What's the difference between horizontal and vertical scaling, in one sentence each?

MEDIUM

RAPID

Vertical scaling adds more resources (CPU/RAM) to a single existing instance. Horizontal scaling adds more instances and distributes load across them — generally preferred for Node given its single-threaded-per-process execution model.

84. What does "fail open" vs "fail closed" mean for a security control, e.g. a rate limiter whose backing Redis goes down?

HARD

RAPID

"Fail open" means the system allows requests through when the control itself fails (e.g. rate limiter down → allow all traffic, prioritizing availability). "Fail closed" means it denies by default on failure (prioritizing safety/security). The right choice depends on what the control protects — auth checks generally fail closed; a rate limiter often fails open to avoid a full outage from a non-critical dependency.

85. What's the risk of using `Math.random()` for generating security tokens or session IDs?

MEDIUM

RAPID

`Math.random()` is not cryptographically secure — its output can be predictable enough to be reconstructed by an attacker in some engines. Use `crypto.randomBytes()` or `crypto.randomUUID()` for anything security-sensitive (tokens, session IDs, password reset codes).

86. What's a common cause of a Node process's file descriptor limit being hit in production, and how do you notice it?

HARD

RAPID

Unclosed sockets, file handles, or database connections that leak over time — often from error paths that skip cleanup logic. Symptom: `EMFILE`/`ENFILE` errors under sustained load, worsening the longer the process runs since it's cumulative, not a one-time spike.

87. Why might you choose UUID v4 over an auto-incrementing integer for a primary key in a distributed system?

MEDIUM

RAPID

UUIDs can be generated client-side or by any service instance without coordinating with a central counter, which matters for sharded/distributed writes and avoids a single point of contention. The trade-off: UUIDs are larger (indexing/storage cost) and, as random v4 values, cause worse index locality than sequential IDs.

88. What does "at-least-once" vs "exactly-once" delivery mean for message queues, and which is actually achievable in practice?

HARD

RAPID

At-least-once guarantees delivery but allows duplicates (achievable, and what most real systems provide). Exactly-once is what people usually want but is extremely hard to guarantee end-to-end across a distributed system in the general case — in practice, "effectively exactly-once" is achieved via at-least-once delivery plus idempotent consumers, not a true single-delivery guarantee at the transport layer.

89. What's the difference between `res.send()`, `res.json()`, and `res.end()` in Express?

MEDIUM

RAPID

`res.json()` serializes the argument to JSON and sets the appropriate content-type header. `res.send()` is more general — it infers content type based on the argument's type (string, buffer, object). `res.end()` is the lowest-level, ends the response with optional raw data and no content-type inference at all.

90. What is "graceful degradation" in a system-design context, with a concrete backend example?

MEDIUM

RAPID

Continuing to serve core functionality when a non-critical dependency fails, instead of failing the whole request. Example: an e-commerce product page whose "recommended items" section depends on a recommendation service — if that service is down, render the page without recommendations rather than failing the entire page load.

91. Why is comparing two floating-point currency values with `===` considered bad practice, and what's the standard fix?

MEDIUM

RAPID

IEEE-754 float representation error means arithmetic results that should be mathematically exact (like `19.99 * 3`) can differ by tiny fractions from the expected value. Store and compute in integer minor units (cents) or use a fixed-precision decimal library, never raw floats, for money.

92. What's the practical difference between `npm ci` and `npm install` in a CI/CD pipeline, and why does it matter?

MEDIUM

RAPID

`npm ci` installs strictly from `package-lock.json` (fails if it's out of sync with `package.json`) and always starts from a clean `node_modules`, giving fully reproducible, deterministic builds — the correct choice for CI. `npm install` can update the lockfile and is meant for local development, not automated pipelines where reproducibility matters.

END OF BOOK

You Now Know Where the Bodies Are Buried

Not literally — but you know where the event loop hides its footguns, where memory leaks actually come from, and how to defend every one of those answers under follow-up questions. That's the actual bar for a senior backend interview.



BACKEND MASTERY SERIES • EDITION 1.0